

Szegedi Tudományegyetem

Informatikai Intézet

Szoftverfejlesztés Tanszék

SZAKDOLGOZAT

Germán Martin

2025

**Szegedi Tudományegyetem
Informatikai Intézet**

Mentálhigiéniai webapplikáció

Szakdolgozat

Készítette:

Germán Martin

üzemtechnológus-informatikus
szakos hallgató

Témavezető:

Dr. Jász Judit

egyetemi adjunktus

Szeged
2025

TARTALOMJEGYZÉK

TARTALMI ÖSSZEFOGLALÓ	1
BEVEZETÉS	2
FELADATKIÍRÁS.....	3
1. Felhasznált technológiák.....	4
1.1. Vite.....	4
1.2. React.js könyvtár.....	4
1.3. TailwindCSS	4
1.4. Node.js és Express.js	4
1.5. MySQL & phpMyAdmin.....	4
1.6. Autentikáció és biztonság	4
1.7. Axios	4
1.8. Nodemailer és EmailJS	4
1.9. Harmadik féltől származó NPM csomagok.....	4
1.10 Gemini API	5
1.11 Node Package Manager	5
2. Megvalósítás.....	6
2.1. Adatbázis.....	6
2.2. Projekt felépítése	7
3. Backend	8
3.1 Config	8
3.2 Controllers	9
3.3 Middleware	10
3.4 Routes	12
3.5 Belépési pont	13
4. Frontend	14
4.1 Oldalak és navigáció.....	16
4.2 Animációk	17
4.3 Modulok és komponensek.....	18
4.3.1 Bejelentkezés.....	19
4.3.2 Chatasszisztens - Hénia.....	20
4.3.3 Meditáció – Meditation Room.....	22
4.3.4 Tanulás – Learning Hub.....	24
4.3.5 Napló – My journal	26
4.3.6 Cikkek – Articles	28
4.3.7.1 Közösség – Community Space.....	31
4.3.7.2 Profil.....	34
5. Továbbfejlesztési ötletek	35
Irodalomjegyzék	37
Nyilatkozat	40
Melléklet	41

TARTALMI ÖSSZEFOGLALÓ

- **A téma megnevezése:**

Mentálhigiéniai webalkalmazás

- **A cél megfogalmazása:**

A szakdolgozatom célja egy modern és innovatív webalkalmazás fejlesztése, amely kényelmes felületen, többféle funkcióval támogatja a felhasználókat mentális egészségük megőrzésében és fejlesztésében.

- **Indoklás:**

Az elmúlt években a mentális egészség kérdése egyre nagyobb figyelmet kapott, különösen a fiatalok körében. Nemzetközi és hazai kutatások is rámutatnak, hogy a stressz és szorongás előfordulása növekvő tendenciát mutat. A digitális eszközök és webalkalmazások lehetőséget kínálnak arra, hogy támogató környezetet biztosítsanak a felhasználóknak, segítve őket a mindennapi kihívások kezelésében.

- **Alkalmazott eszközök, módszerek:**

A szakdolgozatban egy modern, reszponzív webalkalmazás került megvalósításra. A frontend fejlesztés React.js és TailwindCSS segítségével történt, míg a backend Node.js + Express.js alapokon épült, MySQL adatbázissal.

- **Konklúzió:**

A projekt során sikerült számos kulcsfontosságú technológiát és módszert megvalósítani, mint például a monorepo struktúra kialakítása, a választott technológiák integrálása, Google fiókkal való belépés és email cím hitelesítés, animációk, chatbot és számos API implementálás, valamint hangfájlok időzítőre történő lejátszása. Ezek a funkciók hozzájárulnak ahhoz, hogy a fiatalok és fiatal felnőttek hatékonyan kezeljék a stresszt, támogassák tanulásukat és közösségi kapcsolataikat. A projekt technológiai megvalósítása modern, iparági szinten releváns eszközökre épül, így a gyakorlati hasznosság mellett szakmai értéket is képvisel.

- **Kulcsszavak:**

Mentálhigiénia, React.js, Node.js, MySQL, webalkalmazás

BEVEZETÉS

Az elmúlt években a mentális egészség kérdése egyre nagyobb figyelmet kapott a közösségi médiában, ezért mondanom sem kell, hogy különösen a fiatalok körében lett meghatározó téma. Ennek hatására én is egyre többet kezdtem el foglalkozni vele. Megannyi nemzetközi és hazai kutatások is rámutatnak arra, hogy a stressz, a szorongás és a kiégés előfordulása folyamatosan növekvő tendenciát mutat a 12–25 éves korosztályban. Ez a korosztály különösen érzékeny, hiszen az általános iskolás felső tagozattól kezdve a középiskolásokon át egészen a fiatal felnőttekig olyan élethelyzetekkel találkozhatnak, amelyek fokozott terhelést jelentenek: serdülőkor, tanulmányi kötelezettségek, rosszabb esetben iskolai bántalmazások, vizsgák, pályaválasztás, munkahelyi beilleszkedés, valamint a társas kapcsolatok alakulása. A mentális egészség megőrzése és támogatása ezért kiemelt jelentőségű.

A szakdolgozatom témájául egy olyan mentálhigiéniai webalkalmazás fejlesztését választottam, amely modern technológiákra építve nyújt támogatást a felhasználóknak a mindennapi stressz kezelésében, a tanulásban és a személyes fejlődésben. Mivel én is próbáltam mind tanulmányaim, mind a napi élet során az alábbi technikákat és valós életbeli eszközöket, hogy a projekt célja egy olyan platform létrehozása, amely egyszerre kínál önreflexiós lehetőségeket (napló, napi jegyzetek), motivációs tartalmakat (idézetek, történetek), tanulást segítő eszközöket (Pomodoro, ismétlőkártya), valamint relaxációs funkciókat (meditációs hangok, légzés technikák). Emellett közösségi élményt is biztosít egy fórumon keresztül, ahol a felhasználók oszthatják meg tapasztalataikat, és támogatathatják egymást, valamint Hénia (Gemini 2.5 free API) is, aki az oldal chatbot asszisztens-e, akitől beépített prompt opciók közül választva lehet kérdezni, vagy akár saját kérdéseket is feltehetünk neki.

A fejlesztés során törekedtem arra, hogy az alkalmazás ne csupán funkcionálisan legyen hasznos, hanem technológiai szempontból is megfeleljen a modern iparági elvárásoknak. A frontend React.js és TailwindCSS segítségével készült, amely biztosítja a reszponzív és felhasználóbarát felületet, illetve egyéb UI könyvtárakat is használtam. A backend Node.js + Express.js alapokon épült, MySQL adatbázissal, amely a felhasználói adatok, naplóbejegyzések és fórumhozzászólások kezelését, valamint számos egyéb feladatot is ellát. A backend keretrendszernek és az abban található csomagoknak köszönhetően token autentikáció, Google fiókkal való bejelentkezés, input és egyéb validációk egyszerűen megvalósíthatóak voltak. A projekt során kialakítottam egy monorepo struktúrát, amely átláthatóbbá és bővíthetőbbé teszi a fejlesztést, kiemelve, hogy egyetlen start commandal futtatható a backend, illetve frontend.

A szakdolgozatban bemutatott alkalmazás nem csupán egy technológiai gyakorlat, melynek során modern keretrendszerek használatával készítettem egy olyan webapplikációt, hanem egy olyan eszköz, amely valós életbeli problémára ad megoldást, vagyis segíti a felhasználókat a mindennapi kihívások kezelésében motivációval, naplóírással, cikkek olvasásával, támogatja őket a tanulásban és a közösségi kapcsolódásban.

Zárásképpen szeretném összeségében elmondani, hogy a szakdolgozatom célja egy olyan biztonságos, skálázható, bővíthető és felhasználóbarát webalkalmazás létrehozása, amely hozzájárul a felhasználók mentális jólétéhez, és egyben szemlélteti a modern webfejlesztési technológiák gyakorlati alkalmazását. Elkülönítve a felhasználó és az admin szerepkört, melyek különböző jogosultságokkal rendelkeznek alkalmazás szinten. A dolgozatban bemutatott rendszer egyszerre képvisel szakmai értéket, társadalmi és személyes célú hasznosságot, így a fejlesztés nemcsak tanulmányi feladatként, hanem egy valós problémára adott megoldásként is értelmezhető.

FELADATKIÍRÁS

A feladat egy komplex, több modulból álló, reszponzív, biztonságos és bővíthető webalkalmazás megvalósítása, amely egyszerre nyújt relaxációs, tanulást segítő, közösségi és önreflexiós funkciókat. A rendszer célja, hogy olyan digitális környezetet teremtsen, amely a fiatal felhasználók számára könnyen kezelhető, vizuálisan letisztult, mégis technológiailag korszerű megoldásokkal működik. A platformnak teljes körű felhasználókezelést kell biztosítania, beleértve a regisztrációt, a bejelentkezést, az email-es megerősítést, a Google-alapú hitelesítést és a tokenes azonosítást is, miközben külön jogosultsági szintek (vendég, felhasználó és admin) szerint kell szabályoznia a hozzáféréseket. A felhasználóknak lehetőséget kell kapniuk különféle saját tartalmak, például naplóbejegyzések, jegyzetek vagy fórumhozzászólások létrehozására és szerkesztésére, az adminisztrátorok számára pedig biztosítani kell a cikkek kezelését és a moderációs feladatok elvégzését.

A rendszernek tartalmaznia kell egy olyan chatasszisztenst is, amely mesterséges intelligencia alapú kommunikációval segíti a felhasználókat. A személyes tartalmak kezeléséhez egy olyan modul szükséges, amely rich text formátumú naplóbejegyzések létrehozását és szerkesztését teszi lehetővé, megfelelő tartalomtisztítással és biztonságos adatkezeléssel kiegészítve. A bejegyzések megjelenítésének lapozható, vizuálisan esztétikus formában kell történnie, miközben az adatokat egy háttérben futó REST API-n keresztül, MySQL adatbázisban kell tárolni.

A tanulást támogató modulnak olyan funkciókat kell biztosítania, amelyek időgazdálkodási és memorizálási technikákat segítenek. Ennek része a kétféle beépített pomodoro módszer, háttérzenei és hangjelzési lehetőségekkel, valamint egy tanulókártya rendszer, amely statisztikákkal és grafikus visszajelzésekkel támogatja a felhasználót. A modulnak meg kell őriznie a személyes beállításokat, így biztosítva az egyedi felhasználói élményt. Emellett a meditációs és légzéstechnikai funkciók időzíthető hangalapú gyakorlatokat és különböző nehézségi szintű, animált légzésmintákat is magukban foglalnak, reszponzív grafikai megoldásokkal és korszerű animációs technikákkal megvalósítva.

A közösségi felület olyan fórummodult foglal magában, ahol kategóriák szerinti bejegyzések hozhatók létre, amelyekhez a felhasználók hozzászólhatnak, szavazhatnak rájuk, és szükség esetén jelenthetik is azokat. A moderációs lehetőségek és az adatkezelés szintén a backend API-n és az adatbázison keresztül működnek. A teljes frontend React-alapokra épül, komponensstruktúrával, TailwindCSS által biztosított reszponzív kialakítással, valamint modern animációs megoldásokkal kiegészítve. A feladat része egy átlátható rendszerarchitektúra kialakítása, amelyben a frontend és a backend elkülönülten, mégis egységes monorepo környezetben működnek. A MySQL adatbázis feladata a felhasználói adatok, bejegyzések, fórumtartalmak és egyéb metaadatok strukturált és biztonságos tárolása.

A fejlesztés során biztosítani kell a biztonságos adatkezelést, az érvényesítést és a megelőző védelmi mechanizmusokat, valamint a megbízható, REST alapú kliens–szerver kommunikációt. A felhasználói élmény kiemelt figyelmet igényel, így a rendszernek modern UX- és UI-elvek szerint kell működnie, átlátható, újrafelhasználható és moduláris kódszerkezettel. A végső alkalmazásnak egy teljes mértékben működőképes, tesztelt és dokumentált webes rendszernek kell lennie, amely nemcsak technológiai demonstrációként állja meg a helyét, hanem valós igényeket is kielégít: támogatja a felhasználók mentális jóllétét, tanulását, önreflexióját és közösségi kapcsolódási lehetőségeit.

1. Felhasznált technológiák

1.1. Vite

A Vite [1] egy modern build eszköz, amely gyors fejlesztői élményt nyújt. A Hot Module Replacement funkcióval a változtatások azonnal megjelennek a böngészőben, nem kell újra futtatni vagy frissíteni a frontendet, így a fejlesztés gördülékenyebb.

1.2. React.js könyvtár

A React [2] egy nyílt forráskódú JavaScript könyvtár, amely komponens-alapú felépítést biztosít a felhasználói felületekhez. A projektben a React felel a responszív és dinamikus frontend működéséért.

1.3. TailwindCSS

A TailwindCSS [3] egy CSS keretrendszer, amely gyors és rugalmas stílusbeállításokat tesz lehetővé. A responszív design kialakításában nagy szerepet játszott.

1.4. Node.js és Express.js

A Node.js [4] egy JavaScript futtatókörnyezet, míg az Express.js [5] egy minimalista keretrendszer, amely egyszerűvé teszi a REST API-k létrehozását és az útvonalak kezelését. A backend logika és az adatbázis-kezelés ezen alapult.

1.5. MySQL & phpMyAdmin

A MySQL [6] egy relációs adatbázis-kezelő rendszer, amely adatokat táblákban tárol, és támogatja a táblák közötti kapcsolatok létrehozását. A phpMyAdmin egy webes felületet biztosít az adatbázis kezeléséhez, így nincs szükség a parancssoros műveletek használatára.

1.6. Autentikáció és biztonság

Az alkalmazásban JWT [7] (JSON Web Token) alapú autentikációt valósítottam meg. Google fiókos bejelentkezést. A jelszavakat titkosított formában tároltam, valamint injekció elleni védelmet is bevezettem, input validáció és prepare stament függvények segítségével.

1.7. Axios

Az Axios [8] egy népszerű JavaScript-csomag, amely HTTP-kérések kezelésére szolgál. Mivel Promise-alapú, támogatja az aszinkron műveleteket, így az adatok csak a későbbiekben válnak elérhetővé.

1.8. Nodemailer és EmailJS

Az EmailJS [9] és a Nodemailer [10] lehetővé tették, hogy az alkalmazás email küldési funkciókat valósítson meg, például kapcsolat űrlap beküldése vagy regisztráció után email cím hitelesítése.

1.9. Harmadik féltől származó NPM csomagok

- **UI és komponens könyvtárak:**
 - [11]mui/material, @mui/lab – Material UI komponensek
 - [12]shadcn-ui - ikonok, dialógusok, menük, form elemek
- **Animációk és vizuális effektek:**
 - [13]framer-motion – animációk
 - [14]aos – görgetés alapú animációk

- [15]tsparticles, @tsparticles/react – részecske animációk
- **Grafikonok:**
 - [16]react-chartjs-2 – grafikonok készítésére alkalmas
- **Szövegszerkesztők és markdown:**
 - [17]react-quill – rich text editorok
 - [18]react-markdown – markdown megjelenítés
- **Segédkönyvtárak:**
 - [19]moment – dátumkezelés
 - [20]dompurify – HTML sanitizálás
 - [21]js-cookie, [22]cookie-parser – sütik kezelése
 - [23]cors – cross-origin kérések kezelése
- **Adatbázis és titkosítás:**
 - [24]mysql2 – adatbázishoz való csatlakozásért felel
 - [25]bcrypt – jelszó titkosításért felelő algoritmusokat tartalmazó könyvtár
- **Hitelesítés és Google autentikáció:**
 - [26]react-oauth/google
 - [27]google-auth-library
- **Navigáció:**
 - [34] react-router-dom
- **Egyéb funkciók:**
 - [28]multer – fájlok feltöltése és tárolása
 - [29]jsonwebtoken, [30]jwt-decode – tokenek kezelése
 - [31]dotenv – környezeti változók

1.10 Gemini API

A Gemini API [32] a Google által fejlesztett mesterséges intelligencia interfész. Ennek az API-nak a segítségével integráltam a projektben a beépített chatbot modult, amely előre beépített promptok segítségével, interaktív módon válaszol a felhasználók kérdéseire, és segíti őket az alkalmazás funkcióinak megismerésében, valamint egyedi felhasználásra is alkalmas.

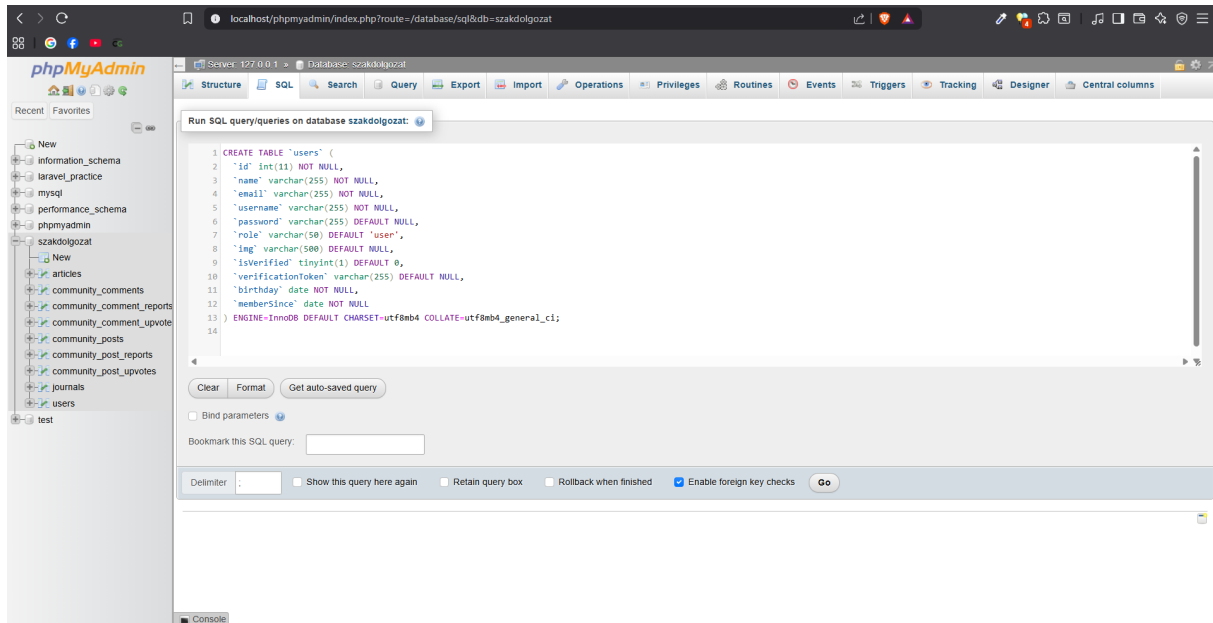
1.11 Node Package Manager

Az NPM [33] a Node.js-hez tartozó csomagkezelő, amivel könnyen lehet külső könyvtárakat telepíteni és kezelni. A projekt során ezzel oldottam meg a szükséges függőségek hozzáadását. A package.json fájlban minden csomag nyilvántartva van, így átlátható és egyszerű a verziók kezelése.

2. Megvalósítás

2.1. Adatbázis

A projectemben egy MySQL [6] alapú relációs adatbázist alkalmaztam, melynek elsődleges célja az azonosításra szolgáló és a felhasználók által tárolni kívánt tartalmak, biztonságos, strukturált tárolása. Az adatbázis teljes egészében phpMyAdmin felületén keresztül, natív SQL lekérdezésekkel került kialakításra. A táblák létrehozása, módosítása és az idegen kulcsok beállítása manuálisan történt, nem használtam semmilyen migrációs rendszert, míg a rekordok CRUD műveletek logikai megvalósításaért a backend felel.



1. ábra
USER tábla létrehozása

Az adatbázisban több, egymással kapcsolatban álló tábla található, amelyek lefedik az alkalmazás fő funkcióit:

- **users:** a felhasználók alapadatait tárolja.
 - id (int), name (varchar), username (varchar), password (varchar), role (varchar), img (varchar), isVerified(tinyint), verificationToken (varchar), birthday (datetime), created_at (timestamp)
- **articles:** a cikkek és írások tárolására szolgál, amelyekhez a szerző azonosítója idegen kulccsal kapcsolódik a users táblához.
 - id (int), title (varchar), desc (text), img (varchar), cat (varchar), created_at (timestamp)
- **journals:** a felhasználók saját naplóbemjegyzéseit tartalmazza.
 - id (int), title (varchar), content (text), created_at (timestamp)
- **community_posts:** a fórum bejegyzéseit tárolja, idegen kulcsokkal biztosítva a kapcsolódást a felhasználókhoz és egymáshoz.
 - id (int), user_id (int), category (varchar), title (varchar), content (text), created_at (timestamp)

- `community_comments`: a fórum hozzászólásait tárolja, idegen kulcsokkal biztosítva a kapcsolódást a felhasználókhöz és egymáshoz.
 - `id` (int), `post_id` (int), `user_id` (int), `content` (text), `parent_comment_id` (int), `created_at` (timestamp)
- `community_comment_upvotes`: a közösségi funkcióhoz tartozó kommentekre érkező szavazatokat kezelik, ennek megegyezően a `community_comment_reports` a jelentéseket. Egyedi kulcsokkal biztosítva, hogy egy felhasználó ugyanarra a tartalomra csak egyszer szavazhasson vagy jelentést tehessen.
 - `community_comment_upvotes`
 - `id` (int), `comment_id` (int), `user_id` (int), `reason` (text)
 - `community_comment_reports`
 - `id` (int), `user_id` (int), `comment_id` (int)
- `community_post_upvotes`: a közösségi funkciókhoz tartozó bejegyzésre érkező szavazásokat kezelik, ennek megegyezően a `community_post_reports` a jelentéseket. Egyedi kulcsokkal biztosítva, hogy egy felhasználó ugyanarra a posztra csak egyszer szavazhasson vagy jelentést tehessen.
 - `community_post_upvotes`
 - `id` (int), `user_id` (int), `post_id` (int)
 - `community_post_reports`
 - `id` (int), `user_id` (int), `post_id` (int), `reason` (text)

Az idegen kulcsok használata biztosítja az adatok integritását, például, ha egy felhasználó törlésre kerül, akkor a hozzá tartozó bejegyzések és hozzászólások is automatikusan törlődnek (ON DELETE CASCADE). Ezáltal az adatbázis konzisztens marad, és nem keletkeznek üresen hagyott, árva rekordok.

Az adatbázis kapcsolat egy külön modulban van kialakítva `db.js`, amelyhez a `mysql2` [24] csomaggal hozok létre kapcsolatot. Itt környezeti változókat használok: a `host`, felhasználónév, jelszó, adatbázis neve és port mind a `.env` fájlból töltődik be, így ezek nincsenek kódban „beleégetve”.

A kapcsolat létrehozása egyszerű: a `mysql.createConnection()` egy aktív kapcsolatot készít, amit `db` néven exportálok. Ezt az egész projekt használja a kontroller függvényekben, amikor SQL lekérdezéseket kell futtatni, így egy helyen van definiálva az adatbáziskapcsolat, átlátható és könnyen karbantartható.

2.2. Projekt felépítése

A projekt során `npm` [33], azaz `node package manager`-t használtam, ami a harmadik féltől származó csomagokat, könyvtárakat tartalmazza. Minden csomagot szükséges az `npm install` terminálba kiadott kulcsszóval telepítenem, ezért a dolgozat írása során, külön nem térek ki erre. A filestruktúrája kettő fő részből áll, backend és frontend.

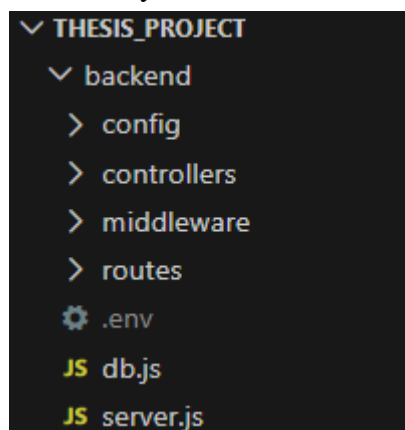
A backend felel az alkalmazás szerveroldali logikájáért, az adatbázis-kapcsolatokért, valamint a biztonságért. Itt történik a felhasználók azonosítása, a CRUD műveletek (létrehozás, olvasás, módosítás, törlés), valamint az adatok validálása és feldolgozása. A backend biztosítja

az API-végpontokat express-route-okon, amelyeken keresztül a kliens kommunikál a szerverrel. A frontend a felhasználói felületet valósítja meg. Ez az a rész, amivel a felhasználó közvetlenül találkozik. Az oldalak, komponensek, gombok, űrlapok és vizuális elemek. A frontend felelős a reszponzív design kialakításáért, a statikus és/vagy dinamikus megjelenítésért, az interaktív funkciókért, valamint a felhasználói élmény biztosításáért és a háttérben futó szerver felé irányított kérések küldéséért. Míg a vékony backend, vastag frontend logikát követtem, a kliensoldali logika a backend által szolgáltatott adatokra épül (REST API), így a két réteg szorosan együttműködik.

Sikerült egy monorepo struktúrát kialakítanom, amely lehetővé teszi, hogy a gyökér könyvtárból egyetlen terminálba írt indító parancs (`npm run dev`) hívásával futtatni tudjam a klienst és a szervert. Ennek megvalósítása közös `package.json`, `vite.config.js` és egyéb konfigurációkkal történt.

3. Backend

A project szerveroldali része a `backend/` mappában található. Itt helyezkedik el az alkalmazás teljes háttér logikájának, adatkezelésének, valamint az adatok biztonságos kezelésének a megvalósítás. A felépítés moduláris, minden funkció külön mappában és fájlban került, így a rendszer átlátható, karbantartható és könnyen bővíthető.



2. ábra
Backend fájlstruktúra

3.1 Config

A `config/` mappában található az `envConfig.js` fájl, amely a környezeti változókért felelős `dotenv` fájl elérési útvonaláért felel, backend oldalon. Erre azért volt szükség, mivel a monorepo kialakítása során a gyökér könyvtárat vette kiinduló pontként, így futtatáskor nem tudta kiolvasni megfelelően a környezeti változók értékét, ezért hibával szállt el a backend kapcsolódásakor, amit a `config` fájl megvalósítása orvosolt. Ezeket az értékeket a `backend/.env` fájlból importálom, így az érzékeny adatok (például adatbázis kapcsolat, JWT titkos kulcs, Google kliens azonosító és egyebek) nincsenek közvetlenül a kódban tárolva. Ez biztonsági szempontból kiemelten fontos, hiszen az ilyen érzékeny információk kiszivárogtatása, vagyis például a forráskódban való tárolásuk, sérülékennyé teszi a rendszert, ezért ezeket jobb egy titkosított, segédfájlban tárolni, ezzel elkerülve az adatok illetéktelen kézbe való jutását.

3.2 Controllers

A kontrollerek szerepe az alkalmazás során, hogy itt van megvalósítva minden üzleti logikája az alkalmazásnak. Ez felel a biztonságért (input validációk, prepare statmentek), a felhasználó által küldött kérések ebben a rétegben kerülnek feldolgozásra, összeköti a klienst és a szervert, és innen történik a kommunikáció az adatbázis vagy más háttérszolgáltatások, külső API-k felé. Kialakítás során, még ha a projectem nem is követi teljesen, de figyelembe vettem az MVC architektúrát, és próbáltam a fájlszerkezetet ennek megfelelően alakítani. Ez a legátláthatóbb és leglogikusabb kialakítás, a hosszabb karbantarthatóság érdekében. A `controllers/` mappában minden fő funkcióhoz külön controller tartozik, így jól elkülönülnek az egyes modulokhoz tartozó műveletek és logikai lépések.

`articleController.js` a cikkek kezeléséért felelős CRUD (CreateReadUpdateDelete) műveleteket valósítja meg. Létrehozás, olvasás, módosítás, törlés. A controllerben található például egy `addPost()` metódusa, amely egy konstansban definiált, előkészített SQL lekérdezés string, hozzáad egy új cikket az adatbázishoz. A `query prepare stament`, vagyis a kérdőjeles helyettesítő paraméterek, és mellette egy tömbben felsorolva az érték, nem pedig a query-stringbe, beégetve való tárolás megakadályozza, az SQL injekciót, valamint input validáció is megtörténik a `req` objektumból értékek feldolgozása során. Hiba esetén pedig visszatérünk az 500 státuszkóddal és a hibaüzenettel, siker esetén pedig egy JSON objektumban visszaküldjük a sikeres üzenet feliratot.

```
export const addPost = (req, res) => {
  const q = `INSERT INTO articles (title, `desc`, img, cat, date,
uid) VALUES (?)`;

  const values = [
    req.body.title, req.body.desc, req.body.img,
    req.body.cat, req.body.date, req.user.id,
  ];

  db.query(q, [values], (err, data) => {
    if (err) {
      console.error("SQL error:", err.message);
      return res.status(500).json(err);
    }
    return res.json("Post has been created.");
  });
};
```

A kontrollereken tovább haladva, mindben megtalálhatóak nagyjából ugyan azon logikán alapuló input validáció, prepare statementek használata az SQL lekérdezéshez, valamint a hibakezelés. Ez biztosítja, hogy az adatok biztonságba kerüljenek az adatbázisba, és megakadályozzuk a támadást.

- `authController.js` – a felhasználói regisztráció, bejelentkezés, email-verifikáció és JWT token generálás logikáját tartalmazza.
- `communityController.js` – a fórum posztok és kommentek kezelését végzi, ide tartozik a hozzászólások hierarchiája, szavazások és jelentések.
- `journalController.js` – a naplóbejegyzések CRUD műveleteit valósítja meg.
- `uploadController.js` – a fájlok (például profilképek) feltöltését kezeli.
- `UserController.js` – a felhasználói profilokhoz kapcsolódó funkciókat tartalmazza (adatok lekérése, módosítása).

Az adatok frissítése (update) esetén, illetve ott, ahol szükséges volt, `async/await` műveleteket használtam. Az `async`, `await` kapcsolatnak a lényege, hogy ez egy Promise-alapú műveleteket szinkronhoz hasonló módon tudom kezelni, vagyis egy ígéretet vár, request formájában, arra pedig egy választ, responseként. Az `async` felel azért, hogy amíg nem érkezett válasz a kérésre, addig a felhasználói felület ne legyen blokkolva, ne fagyjon be és a továbbra is használható legyen az alkalmazás grafikus interfésze.

```
export const updateUserProfile = async (req, res) => {...}
```

A jelszavak titkos tárolásáért a `bcrypt` könyvtár felelt, melyet npm csomagként kellett telepíteni, és ezt a `UserController.js`-ben importálni. Ez a könyvtár egy kifejezetten jelszavak biztonságos hash-elésére kifejlesztett algoritmus. A `bcrypt` célja, hogy a felhasználó jelszava soha ne kerüljön bárki számára olvasható formában az adatbázisban, hanem annak egy speciális, úgynevezett egyirányú, szózott hash változata, mely egy olyan visszafejthetetlen formátummá alakítja, amelyet akkor sem lehet megszerezni, ha az adatbázis illetéktelen kezekbe kerül.

```
import bcrypt from "bcrypt";
```

Felhasználni pedig az alábbi módon tettem. (kódrészlet a `UserController.js`, `updateUserProfile` metódusból)

```
if (newPassword) {
  const salt = bcrypt.genSaltSync(10);
  const hashedPassword = bcrypt.hashSync(newPassword, salt);
  query += ", password = ?";
  values.push(hashedPassword);
}
```

A `salt` egy véletlenszerű karakterláncot generál elkerülve, hogy ugyanazt a jelszót adja meg két felhasználó és egyéb védelmeket is biztosít.

A `hashedPassword`, miután „megvan szóva” a jelszó, egy egyirányú hash-t készít, majd ez kerül mentésre az adatbázisba. Melynek végeredménye általában egy nagyon hosszú, értelmezhetetlen karakterláncsorozat például: `$2a$10$EixZ7yseEKbD/sp/8ljtme5P7...`. Ennek visszafejtése a valódi jelszóra gépek számára is évekig, évszázadokig vagy akár tovább is tarthatna, mivel a `bcrypt` szándékosan egy nagyon lassú algoritmus, ezért egyetlen hash előállításához is több tíz-száz miliszekundumot vesz igénybe. A brute-force próbálkozások extrém időigényesek, ezért egy hosszabb, összetett jelszó esetén a teljes keresési tér olyan hatalmasra nő, hogy annak végig próbálása reálisan gyakorlatilag tehát lehetetlen.

Bejelentkezés során létrejön egy token, jwt (Json Web Token) alkalmazásával, melynek 7 napos lejáratot szabtam meg, ezzel biztosítva, hogy a felhasználónak ne kelljen minden egyes alkalommal ki és be jelentkeznie, hanem ezt elegendő 7 naponta megtenni, addig a rendszer megőrzi a tokenjét, de biztonsági okokból hetente meg kell ismételni. Részlet a `authController.js`, `login` metódusából:

```
const token = jwt.sign(
  { id: user.id },
  process.env.JWT_SECRET,
  {
    expiresIn: "7d",
  }
);
```

3.3 Middleware

A modern webfejlesztésben a middleware réteg kulcsfontosságú szerepet játszik az alkalmazások felépítésében, hiszen ezen a rétegen keresztül kerül levédésre a kliens-szerver között futó kérés-válasz folyamat. A middleware-eket a legtöbb esetben a route-okon keresztül

szokás beépíteni az alkalmazásba, így minden kérés áthalad rajtuk, az pedig kezeli az ismétlődő biztonsági vagy előfeldolgozó feladatokat mielőtt a vezérlés eljutna az adott kontrollerhez. Ennek nagy előnye, hogy nem kell minden API végpontban újra megírni a logikai elemeket, helyette központilag, egységes formában implementálható és használható. Ez jelentősen átláthatóbb, karbantarthatóbb, valamint biztonságosabb kódszerkezetet eredményez.

A middleware-ek különösen fontosak, olyan alkalmazás esetében, mint az enyém is. Ott, ahol a frontend és a backend külön szervereken, külön domaineken és külön portokon futnak. Ilyenkor a böngészők biztonsági szabályai (például a CORS vagy a süti továbbításának tiltása) miatt a szervernek kifejezetten engedélyeznie kell a kommunikációt, ugyanez igaz az autentikációra is. Ilyen esetekben a böngészők alpból blokkolnák a kéréseket, ezért a szervernek egyértelműen meg kell mondania, hogy honnan engedélyezi a hozzáférést, így az alkalmazás képes több domain között biztonságosan kommunikálni. A projectem `middleware/` mappája az alábbi két fontos fájl található. Az első, a `corsMiddleware.js`, mely a CORS (Cross-Origin Resource Sharing) beállításokat kezeli. Mint említettem, az én esetemben külön domain, külön porton futott a két szerver. A backend a `localhost:8800`, míg a frontend a `localhost:5173` használja. A `credentials: true` beállítás lehetővé teszi, hogy a böngésző cookie, session token, auth header és más hitelesítési adatokat is továbbítson.

`origin`: mely domainek érhetik el az API-t.

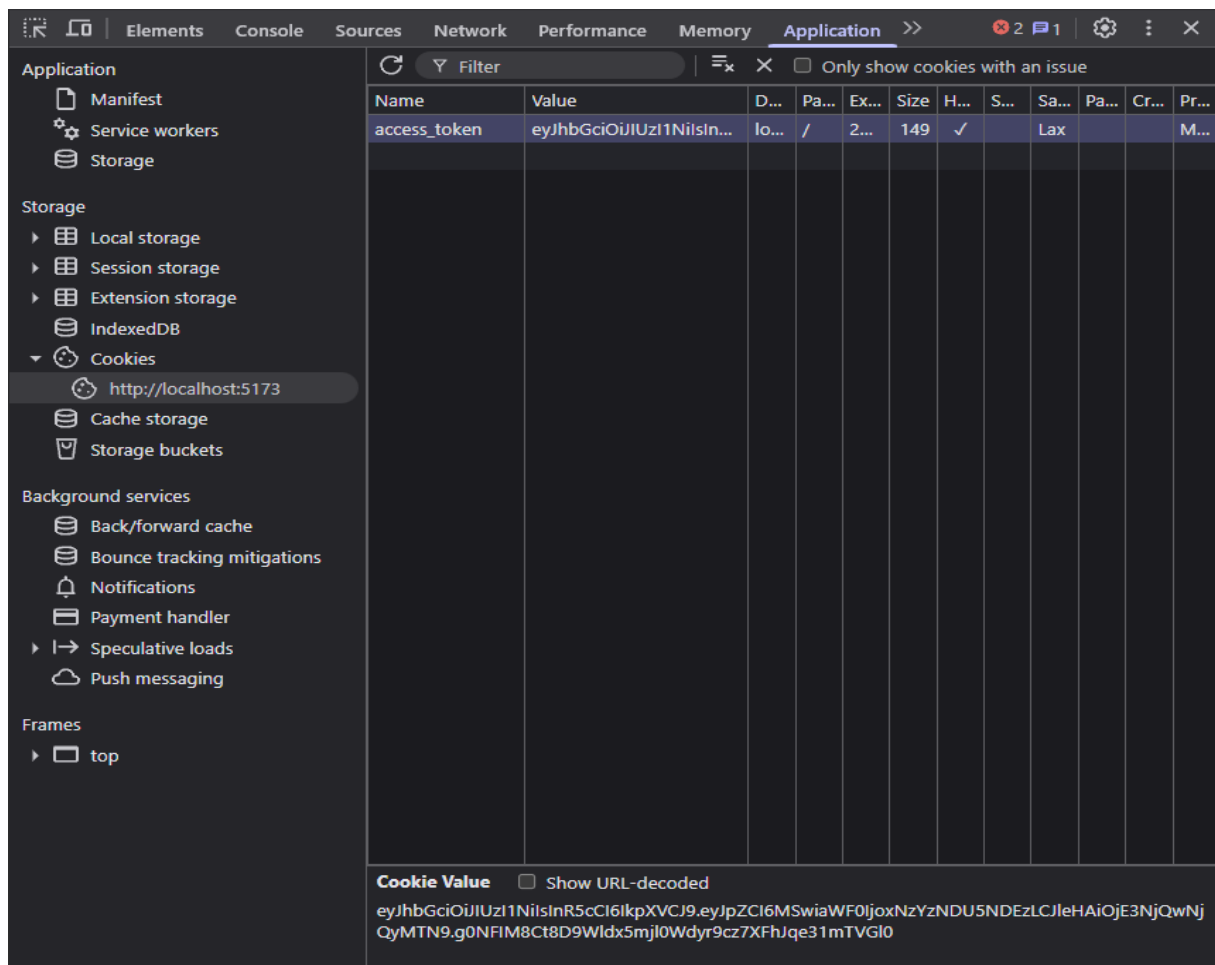
`methods`: milyen HTTP metódusok engedélyezettek (GET, HEAD, PUT...)

`allowedHeaders`: milyen fejléceket küldhet a kliens

és hogy engedélyezett-e a hitelesítés továbbítása

A másik pedig a `verifyToken.js`, amely a JWT token ellenőrzéséért felel. Ez a middleware minden olyan API végpont előtt lefut, amelyhez csak bejelentkezett felhasználó férhet hozzá. A működése egyszerű. Megnézi, hogy a felhasználó rendelkezik-e érvényes JWT tokennel, és ha igen akkor engedi tovább a kérést.

A token a regisztráció vagy bejelentkezés során jön létre, majd egy cookie-ban tárolódik a böngészőben `access_token` néven. Bejelentkezés vagy útvonal váltás során pedig a `verifyToken` middleware ezt a sütit keresi, majd ezt hitelesíti. A `verifyToken` middleware először ezt a cookie-t olvassa ki. Ha nincs ilyen nevezetű cookie, akkor a felhasználó automatikusan 401 - Not authenticated választ kap, vagyis nem hitelesített felhasználó. A `jwt.verify()` metódus ellenőrzi a token valóságát a dotenvben tárolt titkos kulcs (`JWT_SECRET`) alapján. Ha a token hibás, manipulált vagy lejárt, akkor a middleware a 403 státuszkódú `Invalid token or expired` hibát küld vissza. Ha viszont mindent rendben talált, akkor a dekódolt felhasználói adat (`user`) bekerül a `req.user` objektumba. Ezt a későbbi kontrollerek már fel tudják használni, például lekérdezésekhez vagy jogosultság-ellenőrzéshez. Végül, ha minden ellenőrzés sikeres, a middleware meghívja a `next()` függvényt, és átadja a vezérlést a következő lépésnek.



3. ábra
Süti (access_token)

Így működik a teljes autentikációs lánc, és ezzel biztosítható, hogy a védett végpontokhoz csak valós, bejelentkezett felhasználók férjenek hozzá. Amennyiben pedig az urlben átirásra kerülne egy útvonal, mivel a felhasználó nincs bejelentkezve, ezért a home page-re irányítja, ez azonban később, a frontend résznél kerül bemutatásra.

3.4 Routes

A *routes/* mappa az alkalmazás teljes REST API-jának útvonalait tartalmazza. A backend-fejlesztésben az útvonalak szerepe az architektúrában, hogy átjárót képez és összekapcsolják a kliens által küldött HTTP kéréseket (GET, POST, PUT ... stb.) a szerverrel, ezen az útvonalon végighaladó kéréseket pedig a szerveren található megfelelő controllerfüggvényekhez juttassák el, melyek ténylegesen feldolgozzák a kérést. Az útvonalak meghatározzák, hogy a felhasználó milyen erőforrásokhoz férhet hozzá, vagy éppen milyen műveleteket hajthatnak végre és ezeket, milyen jogosultsági feltételek szükségesek az adott végpont eléréséhez. Tehát a kliens által küldött kérés a route-on keresztül halad, a hol a szerver azonosítja az útvonalat, ha be van kötve akkor opcionálisan lefuttatja a middlewaret (akár többet is), majd a vezérlés a megfelelő controller függvényhez továbbítja, amely a tényleges adatfeldolgozást végzi.

A projektben minden fő funkcionális modul külön útvonal-fájlba került, így jól tagolt, áttekinthető, karbantartható struktúra jött létre.

- *articleRoutes.js* – cikkek, bejegyzések listázása, lekérése, hozzáadása, módosítása, törlése.

- `authRoutes.js` – regisztráció, bejelentkezés, google bejelentkezés, jelszóellenőrzés, email-verifikáció.
- `communityRoutes.js` – fórum posztok, kommentek, szavazások, jelentések CRUD műveletei.
- `journalRoutes.js` – személyes naplőbejegyzések teljes CRUD funkciói.
- `uploadRoutes.js` – fájlok (kép, dokumentum stb.) szerverre történő feltöltése, `multer` middleware segítségével.
- `usersRoutes.js` – felhasználói profil adatok lekérése, módosítása és kapcsolódó műveletek.

Próbáltam a megközelítéssel minél jobban megfelelni a REST API-tervezés irányelveinek, hiszen az egyes erőforrásokhoz (articles, users, community stb.) külön nevekkkel, külön http műveletekkel, külön végpontok tartoznak. Minden útvonal-fájl az Express.js keretrendszer `Router()` objektumát használja. Az Express a Node.js legelterjedtebb backend-keretrendszere, amely a Router segítségével lehetővé teszi logikailag elkülöníthető útvonalcsoportok létrehozását. Részlet, a `userRouter.js` tartalmából:

```
import express from „express“;
import { getUserProfile } from „../controllers/userController.js“;
import { verifyToken } from „../middleware/verifyToken.js“;

const router = express.Router();
router.get(“/profile“, verifyToken, getUserProfile);

export default router;
```

A Router objektum technikailag a következő feladatokat látja el.

- csoportosítja az egymáshoz tartozó http metódusokat (GET, POST, PUT, DELETE),
- összeköti a route-okat a megfelelő kontrollerfüggvényekkel,
- middleware-eket kapcsol az útvonalak elé, például hitelesítést vagy validációt,
- elkülöníti a különböző modulok (articles, users stb.) logikáját.

A project útvonalai közül külön kiemelném az `uploadRoutes.js`, mivel ez a modul a szerverre történő fájlok feltöltéséért felel, és egy Node.js middleware-t, a `Multer`-t használja. A `Multer` az Express keretrendszer hivatalos, széles körben használt middleware-e, amely segít a fájlok fogadásában és mentésében, mivel a http-kérések body-ja alapértelmezetten nem képes bináris adatokat kezelni. A `multer` konfigurációjában egy `diskStorage` motor kerül létrehozásra és ez szabályozza, hogy

- `uploadDirectory` – hová kerüljön a mentett fájl. (Nálam ez a `frontend/public/upload`)
- milyen fájlneven kerüljön mentésre. (Nálam ez az eredeti file neve kötőjellel elválasztva + egy időbélyeg `Date.now()`, ezzel elkerülve a fileütközéseket és egyszerűbbé téve az egyedi azonosítást.

Példányosítás után létrejön az `upload` objektum, majd a router egyetlen POST végpontot definiál.

Router nélkül tehát elmondható, hogy minden útvonalat a fő szerverfájlba (`server.js`) kellene írni, ami egy idő után kezelhetetlenné válna és óriási kódbázis eredményezne.

3.5 Belépési pont

A backend legfontosabb részeit pedig a végére hagytam. A `server.js` fájl a backend szerver belépési pontja, ez a szerver „lelke”, enélkül egyszerűen nem lenne mire kéréseket küldenie a kliensnek. Ebben a fájlban indul el minden. Itt jön létre az Express alkalmazás, itt kerülnek

bekötésre a middleware-ek (`cookie-parser`, `corsMiddleware`, `verifyToken`), majd importálom a route-okat és az `app.use()` segítségével regisztrálom őket. A végén az `app.listen()` elindítja magát a szervert, ami innentől fogva képes a böngészőtől érkező kérések fogadására.

A `server.js`-ben töltődnek be a `.env` környezeti változók is, így a projekt futásakor ezek automatikusan betöltődnek, a kód pedig tiszta marad, és környezetenként eltérő beállításokkal is könnyen működik. Miután betöltődtek a middleware-ek és a route-ok, a szerver elindul, a konzolon pedig megjelenik, hogy a backend szerverünk itt melyik szerveren érhető el, erre a címre küldjük a kéréseket.

```
PS C:\Users\marti\OneDrive\Asztali gép\Thesis_Project> npm run dev

> thesis-project@1.0.0 dev
> concurrently "vite" "nodemon backend/server.js"

[1] [nodemon] 3.1.10
[1] [nodemon] to restart at any time, enter `rs`
[1] [nodemon] watching path(s): *.*
[1] [nodemon] watching extensions: js,mjs,cjs,json
[1] [nodemon] starting `node backend/server.js`
[0]
[0] VITE v6.3.5 ready in 554 ms
[0]
[0] → Local: http://localhost:5173/
[0] → Network: use --host to expose
[1] Backend server connected successfully!
[1] Listening at: http://localhost:8800
```

4. ábra
Parancssor az indítóparancs kiadása után

Amennyiben a környezeti változók helyesen töltődtek be és az adatbázis is elérhető, akkor a szerver gond nélkül képes kéréseket fogadni és azokra válaszolni, ezzel megegyezően tárolni és lekérdezni is az adatokat. *(Amennyiben nem monorepo struktúrában dolgoztam volna, úgy a backend könyvtárban a `node server.js` parancsot kellett volna kiadnom.)*

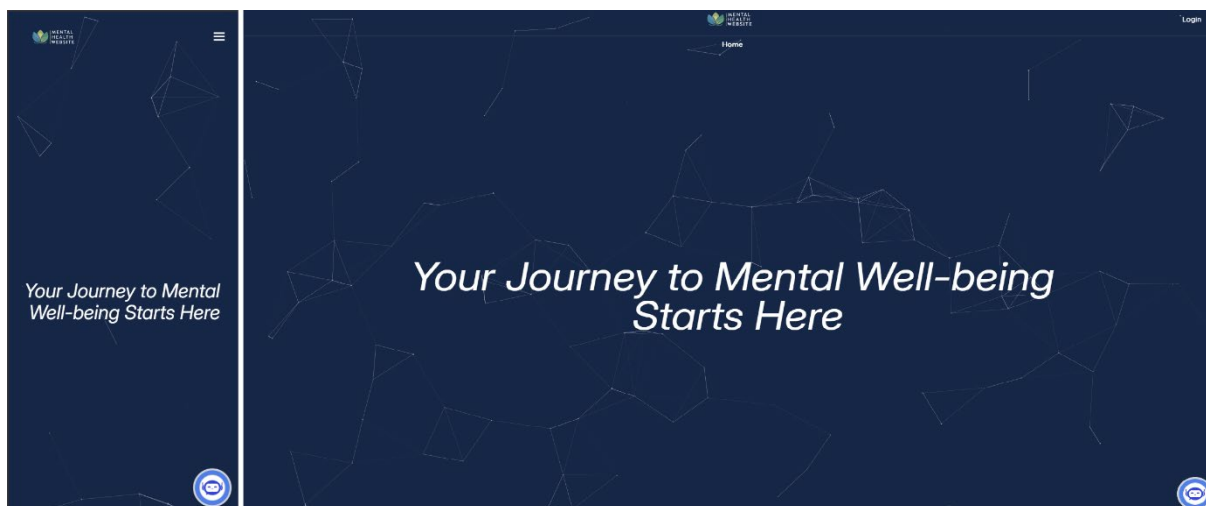
4. Frontend

Az alkalmazásom tervezésekor, mivel frontend közelebb áll hozzám, mint a backend, és a jövőmet is frontend fejlesztőként tervezem, ezért is vastagabb ez a rész, kiválasztáskor fontos szempont volt számomra az, hogy számolva a fejlesztési idővel, mindenféleképpen egy olyan technológiát szerettem volna elsajátítani ez idő alatt, ami hosszú távon is egy értékes tudást biztosít, ezért esett a választás egy keretrendszerre. A keretrendszer kiválasztásánál fontos szempont volt, hogy egy olyan eszközt használjak, amely egyszerre modern, és az általa kínált lehetőségekkel gyorsabb a fejlesztési idő. A React [2] erre ideális megoldást nyújtott, mert a munkaerőpiacon is egyre többet keresett keretrendszer. A fejlesztés során a teljes felület önálló, jól elkülöníthető komponensekre lehetett felbontani. Ez a gyakorlatban azt jelenti, hogy minden oldal vagy funkció például a navigációs sáv, a chatbot ablaka, a kártyák, a bejelentkező felület vagy akár egyetlen gomb, külön fájlban, elkülönítve működik. A komponensek szétválasztásának előnye, hogy egymástól függetlenül fejleszthetők és újra felhasználhatók így, ha valamit módosítani vagy bővíteni kellene rajtuk, nem kell az egész alkalmazást átnézni, elég csak az adott elemnek a komponens forráskódjához elnavigálni.

A React projektben a belépési pontokat a `main.jsx` és az `App.jsx` fájlok jelentik. A `main.jsx` felelős azért, hogy az alkalmazás elinduljon, itt történik meg az App komponens renderelése a `ReactDOM.createRoot` segítségével.

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <GoogleOAuthProvider clientId={clientId}>
      <AuthProvider>
        <AuthContextProvider>
          <App />
        </AuthContextProvider>
      </AuthProvider>
    </GoogleOAuthProvider>
  </StrictMode>
);
```

Az `App.jsx` pedig az alkalmazás fő komponense, amely összefogja az aloldalakat és a közös elemeket (például a navigációs sávot vagy a routert). Itt kerülnek beállításra az útvonalak a `react-router-dom` segítségével, valamint innen indul minden további komponens. A projektet, mint már említettem Vite [1] alapon hoztam létre `npm create vite@latest` paranccsal, ami azért volt előnyös, mert jóval gyorsabb és kényelmesebb fejlesztői környezetet ad, mint a megszokott, alap `npx create-react-app` paranccsal való létrehozás. A Vite gyakorlatilag a Hot Module Replacement funkcióval azonnal újratölti az oldalt módosítás esetén, ezzel sok időt spórolva a fejlesztés során, illetve kisebb és optimalizáltabb pre-buildet állít elő, ami javítja a kész alkalmazás teljesítményét. A felület megjelenéséhez TailwindCSS [3] használtam. Ez a megközelítés teljesen más, mint a hagyományos CSS fájlok írása, mivel a stílusokat közvetlenül az elemekre lehet rárakni rövid, jól áttekinthető utility-osztályok segítségével. Ennek a legnagyobb előnye az volt, hogy nem kellett külön CSS struktúrát kialakítanom, és sokkal gyorsabban tudtam haladni a dizájn kialakításakor. Egyszerűen `npm csomagként` telepíteni kellett a TailwindCSS-t `npm install -D tailwindcss postcss autoprefixer`, illetve minimális `vite.config` és `style.css` beállítás után már a környezet fel is ismerte azt. A Tailwind ráadásul natívan, beépítve támogatja a reszponzív megjelenést, ugyanazzal a komponenssel meg tudtam oldani a mobilos és asztali megjelenést is, csak néhány plusz osztály hozzáadásával.



5. ábra

Kezdőoldal. Baloldalon mobil, jobboldalon asztali verzió

Ez különösen hasznos volt olyan elemeknél, mint a navigációs sáv vagy a lebegő chatbot ablak, amelyeknek teljesen máshogy kell viselkedniük mobilon, mint nagyobb kijelzőkön. A

frontend felépítése során arra törekedtem, UI és UX szempontból minél egyszerűbb, de nagyszerűbb legyen, hogy a felhasználó minél egyszerűbben, „bolyongás” nélkül megtalálja az adott funkciót, de amint megtalálja az minél nagyszerűbb hatást keltsen animációkkal, hover és egyéb stílus elemekkel biztosítva azt.

4.1 Oldalak és navigáció

A frontend egyik legfontosabb feladata, hogy a felhasználót logikusan, átláthatóan és megszakítás nélkül vezesse végig az alkalmazás különböző részein. Mivel a projektben több, egymástól teljesen eltérő funkciót megvalósító oldal található, ezért egy olyan navigációs rendszert kellett kialakítanom, amely egyszerre egyszerű és jól strukturált. Ezt a `React Router` biztosítja, ami a `React` ökoszisztéma szabványos útvonalkezelő könyvtára, és lehetővé teszi, hogy az alkalmazás oldalai valós újratöltés nélkül, úgynevezett kliensoldali útvonalváltással működjenek. Ez jelentősen gyorsítja a felhasználói élményt, hiszen a böngésző nem tölti újra az egész oldalt, csak a ténylegesen változó tartalmat jeleníti meg.

A frontend oldalszerkezete a `frontend/src/pages` mappában található, ahol minden nagyobb funkció külön komponenssel rendelkezik. Az `ArticlePage.jsx` a cikkek megjelenítését kezeli, az `AuthPage.jsx` felel a bejelentkezésért és regisztrációért, a `CommunityPage.jsx` a fórum oldalt tartalmazza, a `JournalPage.jsx` kezeli a naplózási felületet, a `ProfilePage.jsx` a felhasználói adatokat, a `LearnPage.jsx` és `MeditationPage.jsx` pedig különböző tanulás segítő és légzés technikai tartalmakat jelenít meg. A `HomePage.jsx` tölti be a kezdőoldalt, amely animációkkal és bemutatkozó tartalmakkal fogadja a látogatót. A navigáció felépítése az `App.jsx` fájlban történik, ahol elsőként kerülnek inicializálásra a globális kontextusok, majd ezt követően itt definiálom az útvonalakat is. Az `App.jsx` felel azért, hogy melyik URL-n milyen oldal jelenjen meg, és hogy ezekhez az oldalakhoz milyen logika kapcsolódjon. Itt található meg azok a route bejegyzések, amelyek összekötik például a `/articles` útvonalat az `ArticlePage` komponenssel, vagy a `/journal` útvonalat a naplózófelülettel. A `React Router` biztosítja, hogy az útvonalak között váltva ne legyen teljes oldalújratöltés, hanem az oldalak azonnal és zökkenőmentesen jelenjenek meg.

A navigáció egyik fontos kiegészítője a védett útvonalak kezelése. Mivel a projektben több olyan funkció is van, amelyet csak bejelentkezett felhasználók érhetnek el, például a napló, a közösségi felület vagy a saját profil, ezért szükség volt arra, hogy ezeket az oldalakat a rendszer automatikusan levédje. Ennek megoldására hoztam létre a `src/guard/PrivateRoute.jsx` komponenst, amely egyfajta útvonali őrként működik. A komponens először a hitelesítési kontextusból ellenőrzi, hogy a felhasználó be van-e jelentkezve, illetve, hogy a rendszer már betöltötte-e a bejelentkezési állapotot. Ha a betöltés még folyamatban van, akkor egy külön betöltő képernyő jelenik meg `LoadingScreen.jsx`, amely visszajelzést ad a felhasználónak. Ha a felhasználó nincs bejelentkezve, akkor a rendszer automatikusan visszairányítja a kezdőoldalra, és nem engedi megnyitni a védett tartalmat. Ha viszont minden rendben van, a `PrivateRoute` átengedi a felhasználót a kívánt oldalra. Ez a módszer egy biztonságos és felhasználóbarát megoldást ad arra, hogy a rendszer erőforrásait ne is lássa olyan felhasználó, akinek nincs jogosultsága hozzá.

Kódrészlet az `App.jsx`-ből, a `PrivateRoute` bekötésére.

```
<Route path="/post/:id" element={<PrivateRoute><Article /></PrivateRoute>} />
```

A teljes navigáció így három fő rétegből épül fel. A `React Router` alapvető útvonalai, a komponensalapú oldalszerkezet, valamint a védett útvonalak kezelése a `PrivateRoute` segítségével. Ennek köszönhetően az alkalmazás oldalai jól elkülönülnek egymástól, könnyen karbantarthatók, és minden modul saját, logikusan felépített útvonalon érhető el. Ez a felépítés hosszú távon is skálázható, hiszen, ha új funkciót szeretnék hozzáadni, elég csak egy új

oldalkomponenst létrehozni a /pages könyvtárban, majd regisztrálni az útvonalak között az App.jsx-ben.

Minden oldal külön React komponenssel rendelkezik, így nem az oldalak forráskódja nő óriásivá és átláthatatlanná, hanem minimális és letisztult maradt, ezzel is megőrizve a skálázhatóságát és karbantarthatóságát.

```
import HomeComponent from "../components/HomeComponent";
import Note from "../components/utilityComponents/Note";
import Footer from "../components/utilityComponents/Footer";
import Chatbot from "../components/utilityComponents/Chatbot";

function HomePage() {
  const currentUser = JSON.parse(localStorage.getItem("user"));
  return (
    <>
      {currentUser && <Note />}
      <div className="content">
        <HomeComponent />
      </div>
      <div className="footer">
        <Footer />
      </div>
      <Chatbot />
    </>
  );
}

export default HomePage;
```

Például a Home page-hez tartozik egy HomeComponent.jsx fájl, amely több kisebb komponensből tevődik össze, ennek elemei specifikusan meghatározva a HomeComponent mappában találhatóak.

Ennek köszönhetően a fejlesztés során folyamatosan úgy éreztem, mintha LEGO-val játszadoznék, és az egész alkalmazást szépen, átlátható, kisebb darabokból tenném össze. Az alkalmazás során, minden oldalt ennek megfelelően alakítottam ki, törekedve a minél kisebb kódmennyiségre, mindazért, hogy megkönnyítse a fejlesztés jelen és jövőbeli folyamatait, elkülönítve az oldalakat, modulokat és komponenseket egymástól.

4.2 Animációk

Az egész webalkalmazást interaktívra terveztem, ezért az élmény kialakítása is nagyon fontos volt a felhasználók számára. Mivel az alkalmazás témája mentális egészség és önfejlesztés, különösen fontosnak tartottam, hogy a felület ne legyen hirtelen vagy zavaró, hanem finoman reagáljon, „lélegezzon”, és már az első pillanattól kezdve pozitív hangulatot keltsen, megőrizve a felhasználók nyugalmát. Ennek érdekében többféle animációs megoldást használtam a projektben, amelyek különböző területeken segítik a felhasználói élményt. A kezdőoldalon mozgó háttér animáció, görgetéshez kapcsolódó megjelenések, finom átmenetek komponensek között, valamint szöveg animációk.

A kezdőképernyő háttéranimációjához a particles.js [15] könyvtárat használtam, amely egy könnyű, jól konfigurálható JavaScript megoldás interaktív részecske-animációk megjelenítésére. Ez az animáció biztosítja a főoldalon azt a pulzáló, folyamatosan mozgó háttérrel, amely egyszerre modern és vizuálisan megnyugtató. A részecskék finoman mozognak, reagálnak az egér mozgására, és ezzel egy dinamikus, mégis visszafogott atmoszférát adnak az oldalnak. Mivel a particles.js külön komponensben fut, nem terheli túl a többi részt, így az alkalmazás teljesítményére sincs negatív hatása.

A többi oldalon inkább olyan animációkat alkalmaztam, amelyek a felhasználó mozgására, például görgetésre reagálnak. Ehhez az AOS (Animate On Scroll) [14] könyvtárat

használtam, ami egyszerűen integrálható React környezetbe, és képes arra, hogy az elemek csak akkor jelenjenek meg teljesen, amikor a felhasználó ténylegesen odagörget. Ezzel elértem, hogy az oldalak ne egyszerre, hirtelen töltődjenek be, hanem fokozatosan, természetes módon „úszanak be”. Ez nemcsak látványos, hanem UX szempontból is előnyös, mert irányítja a figyelmet és ritmust ad az oldalnak, ezáltal az újonnan érkező felhasználók is kényelmesen feltudják venni az oldalnak ezen ritmusát, és könnyedén felfedezheti és megtanulhatják az oldal vezérlését, valamint a felúszó elemekre egérrel történő irányításával további állapotot érünk el, amivel „nagyobb” lesz az adott komponens. A komponensek közötti finom átmeneteket és kényelmes mozgásokat a `Framer Motion` [13] könyvtárral valósítottam meg. A `Framer Motion` sokkal részletesebb kontrollt ad az animációk időzítése, gyorsulása, elhalása és dinamikája felett, mint a hagyományos CSS átmenetek. Az oldalak váltásánál, gombkattintásoknál, ablak megnyitásánál használtam olyan mozgásokat, amelyek apró, de tudatos részletek. Finom elmozdulás, áttűnés vagy rugós animáció. Ezek együtt sokkal „élőbbé” teszik a felületet. Néhány helyen egy-egy szöveg rész is kapott animációt, mivel a mentális egészséggel kapcsolatos oldalak gyakran tartalmaznak biztató, motiváló vagy érzelmileg fontos üzeneteket. Ezeket nem egyszerűen statikus szöveggént szerettem volna megjeleníteni, hanem mozgással kiemelni őket. Ehhez olyan megoldást használtam, amely lehetővé teszi, hogy a szövegek bizonyos időközönként váltakozzanak, és közben finoman elmozduljanak, elhalványuljanak, majd újra megjelenjenek. Ezek az effektek elsősorban a kezdőlapra kapnak szerepet, ahol a szövegek egyesével „sorolódnak” (loop) ki különböző jelentésekkel és pozitív üzenetekkel, ami már az első pillanatban megadja az oldal hangulatát. Az animációk itt rugós mozgást, átlátszóság és elmosódást használnak, hogy a szövegek természetes, finom ritmusban váltsák egymást.

4.3 Modulok és komponensek

Az alkalmazás teljes működését kisebb, jól elkülöníthető modulokra bontottam, amelyek mind egy-egy konkrét funkcióért felelnek. A modulok a frontend logikai egységei, ilyen például az egyes oldalakhoz tartozó komponensstruktúra, a naplózási felület, a különböző tanulási modulok vagy éppen a fórum funkció. Mivel azonban ezek a modulok csak bejelentkezett felhasználók számára érhetők el, elsőként a navigációban jelenik meg a felosztásuk.

A navigációs sáv, amely lényegében az alkalmazás fő irányítópultja, teljes egészében dinamikus. A rendszer a felhasználó bejelentkezési állapotát figyeli, és csak akkor jeleníti meg a modulokhoz tartozó menüpontokat, ha a hitelesítés már megtörtént. Amíg a felhasználó nem jelentkezik be, a navigáció minimalista módban működik. Kizárólag a főoldal, a bejelentkezés és a regisztráció érhető el. Minden más funkció rejtett állapotban marad. Ezzel azt szerettem volna elérni, hogy a felhasználó már a kezdőoldalon is érezze, hogy a rendszerben sokkal több lehetőség rejtőzik, amelyekhez saját fiókján keresztül tud majd hozzáférni, ösztönözve a felhasználót a regisztrációra. Miután a felhasználó sikeresen regisztrált és bejelentkezik, a teljes menüsor kibővül. Megjelennek a *Home*, *Meditation*, *Learning*, *Journal* és minden további funkció, amely a rendszer részét képezi. Ezek a menüpontok nemcsak egyszerűen láthatóvá válnak, hanem egymás után, `Framer Motion` animáció segítségével jelennek meg. A navigáció elemei rövid késleltetéssel, finom fade-in és slide animációkat kapnak, így a menüsor életre kel, és vizuálisan is jelzi, hogy a felhasználó mostantól a teljes alkalmazást elérheti. Ezzel a megoldással az volt a célom, hogy a bejelentkezés ne csak egy technikai lépés legyen, hanem egyfajta „átlépés” a valódi tartalomba, mintha a felhasználó belépne az alkalmazás saját terébe, ahol minden rendelkezésére áll. A modulokhoz való hozzáférés tehát nem csak technikai feltételekhez kötött, hanem vizuális és UX-szempontból is külön választott állapot. A rendszer így egyszerre marad letisztult és átlátható azok számára, akik még nem rendelkeznek fiókkal, és gazdag funkcionalitást kínál azoknak, akik már bejelentkeztek. Ez a „kétlépcsős” felépítés segít abban is, hogy a felhasználók könnyebben megértsék a rendszer logikáját, a bejelentkezés

után nyílnak meg azok a modulok, ahol a tényleges tartalom, adatkezelés, személyre szabott felület és interaktív funkciók találhatóak. A moduláris felépítésnek további előnye, hogy az alkalmazás bármely része külön fejleszthető, bővíthető vagy akár lecserélhető anélkül, hogy más funkciókat érintene.

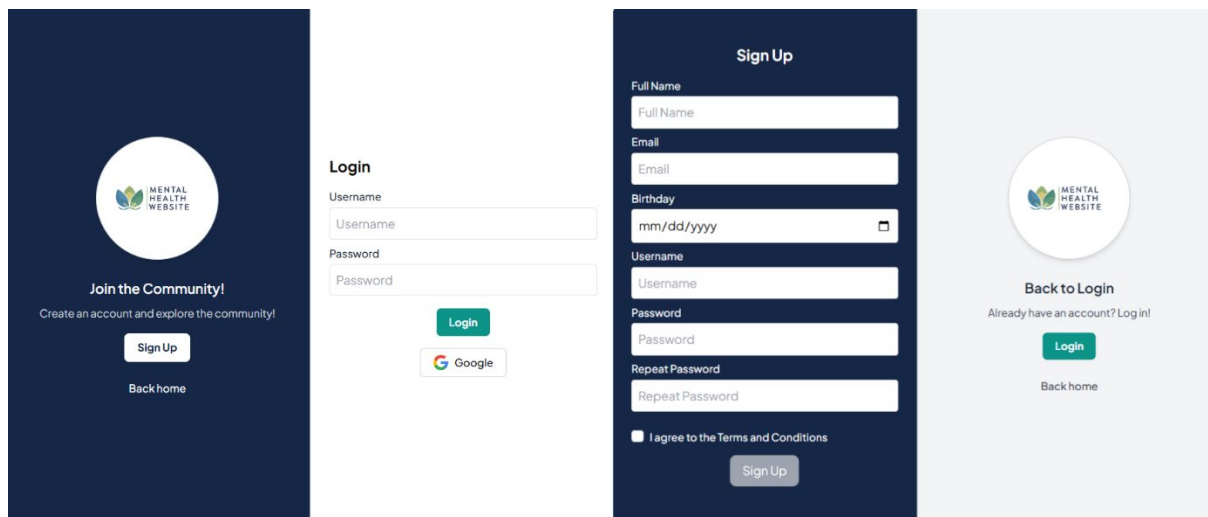
A navigációban megjelenő modulok mögött valójában, mint azt már említettem az oldalak során, külön, önálló oldalkomponensek állnak, amelyek saját belső logikával, saját alkomponensekkel és saját megjelenítési szabályokkal működnek.

4.3.1 Bejelentkezés

Minden alkalmazás során az egyik legfontosabb rész a rendszerhez való hozzáférés, ennek pedig elengedhetetlen része a megfelelő hitelesítés, hiszen az alkalmazásban található funkciók kizárólag bejelentkezett felhasználók számára érhetők el, emiatt a bejelentkezési folyamatot különösen átgondoltan kellett kialakítanom. Egyszerre kellett biztonságosnak, felhasználóbarátnak és technikailag megbízhatónak lennie. A projektben ezt egy külön felületre szerveztem, amely a `/authpage` útvonalon érhető el, és amely a teljes bejelentkezési és regisztrációs logikát kezeli.

A bejelentkezési oldal `React` komponensként működik (`AuthPageComponent.jsx`), és minden hitelesítéssel kapcsolatos működés erről a nézetről indul. A felület egy kártyaszerű elrendezést kapott (`AuthCard` komponens), amely egyszerre kezeli a bejelentkezést és a regisztrációt. A két nézet között egy gombbal lehet váltani, így nincs szükség külön oldalakra a felhasználó ugyanazon a felületen döntheti el, hogy belépni szeretne vagy éppen új fiókot regisztrálna. A váltás logikáját egy egyszerű állapotkezelés végzi (`isRegisterPage`), amely a `useAuthForm` hookból érkezik, és a form működéséért felel. Az `AuthPage` vizuálisan is dinamikus, hiszen az egész komponens `AOS` animációt kapott, így a felhasználó egy finoman „zoom-in” animációval találkozik, amikor megnyitja a felületet. A cél ezzel az volt, hogy már a hitelesítési folyamat is legyen élményszerű, ne pedig csak egy statikus űrlap legyen. A regisztráció folyamata több lépésre lett felosztva annak érdekében, hogy biztonságos maradjon, de továbbra is könnyen használható legyen. A felhasználói adatokat a frontend hitelesíti, tehát ellenőrzi, hogy az adott mezőben valóban a megfelelő típusú adat szerepel-e (szöveg, szám, dátum, jelszó formátum stb.). Minden inputmező kontrollált komponensként működik, vagyis saját állapottal rendelkezik, amelyet a `useState` segítségével kezeltem. Ez a megoldás egyszerű, mégis jól átlátható, és különösen alkalmas olyan űrlapoknál, ahol minden változásra azonnal kell reagálni. A regisztráció csak akkor engedélyezett, ha a felhasználó minden kötelező mezőt kitöltött, és elfogadta a felhasználási feltételeket. Ezt egy `checkbox` jelzi, és amíg nincs kipipálva, a regisztráció gomb `disabled` állapotban marad. Ezzel azt szerettem volna elérni, hogy a felhasználó tudatosan járuljon hozzá a regisztrációhoz, és tisztába legyen a szabályokkal, valamint azok megszegésével járó következményeivel, ne csak „rákattintson” valamire figyelmen kívül. Sikeres regisztráció után a rendszer nem enged azonnal belépni, a felhasználó kap egy megerősítő e-mailt, amelyben található egy link. Erre rákattintva tudja verifikálni a fiókját. A backend ilyenkor az adott felhasználó rekordján beállítja az `isVerified` értéket `true`-ra. Ez azért fontos, mert amíg ez az érték `false`, a rendszer nem engedi bejelentkezni. Ha valaki verifikálatlan fiókkal próbál belépni, egy egyértelmű hibaüzenetet kap, amely figyelmezteti, hogy előbb meg kell erősítenie az e-mail címét. Ezzel biztosítható, hogy a rendszerben ne legyenek olyan felhasználók, akik nem valós e-mail címmel próbálnak belépni. A bejelentkezés kétféle módon történhet.

- Klasszikus e-mail + jelszó kombinációval vagy
- Google Fiókkal.



6. ábra
Bejelentkezés/Regisztráció űrlap

A Google OAuth login a Google által biztosított OAuth2 API-n keresztül működik, ahol a felhasználó adatai (név, e-mail, profilkép) egyenesen a Google-től érkeznek. Ez nagyjában megkönnyíti a hitelesítési folyamatot, mivel a felhasználónak nem kell új fiókot létrehoznia és hitelesíteni azt, hiszen a rendszer ezt automatikusan megteszi. Érzékeli, ha Google-el történt a bejelentkezés, ha igen, automatikusan igaz értéket vesz fel az `isVerified` felhasználói rekord. A frontend csak a Google által küldött tokennel dolgozik, amelyet elküld a backendnek, és ott történik a validálás és a felhasználó adatainak kezelése. A hitelesítési állapotot a teljes alkalmazásban a `AuthContext` kezeli, amely a `React Context API`-ra épül. A context tárolja a `currentUser` objektumot, a betöltési állapotot (`isLoading`), a bejelentkezési és kijelentkezési függvényeket, valamint minden olyan adatot, amely az egész alkalmazás számára releváns, vagyis az alkalmazás több területén, kódszinten tudtam hivatkozni erre az objektumra, és lekérni adatokat amennyiben szükséges volt, valamint tudtam kezelni azok állapotát. A `PrivateRoute` ennek a contextnek alapján dönti el, hogy a felhasználónak szabad-e megnyitnia egy adott oldalt. A hitelesítés eredménye így globálisan, minden komponens számára elérhető, anélkül, hogy propsokon (*properties: a szülő komponens adja át az értékeket a gyermek komponensnek. Nagyobb alkalmazásoknál azonban ez nehézkes lehet, mert minden szinten tovább kell adni az adatokat, akkor is, ha közben nem használják.*) keresztül kellene továbbadni. Ez jelentősen leegyszerűsíti a logikát és stabil alapot ad többoldalas, összetett alkalmazásokhoz.

A bejelentkezési modul tehát nem pusztán egy egyszerű form, hanem egy komplett hitelesítési rendszer űrlapkezeléssel, kliensoldali ellenőrzéssel, e-mail verifikációval, Google OAuth támogatással, globális context alapú állapotkezeléssel, animált felhasználói élménnyel, valamint védett útvonalakkal. Ezek együtt biztosítják azt, hogy a felhasználó biztonságosan, kényelmesen és modern környezetben tudjon belépni az alkalmazásba, és hozzáférhessen a fő funkciókhoz.

4.3.2 Chatasszisztens - Hénia

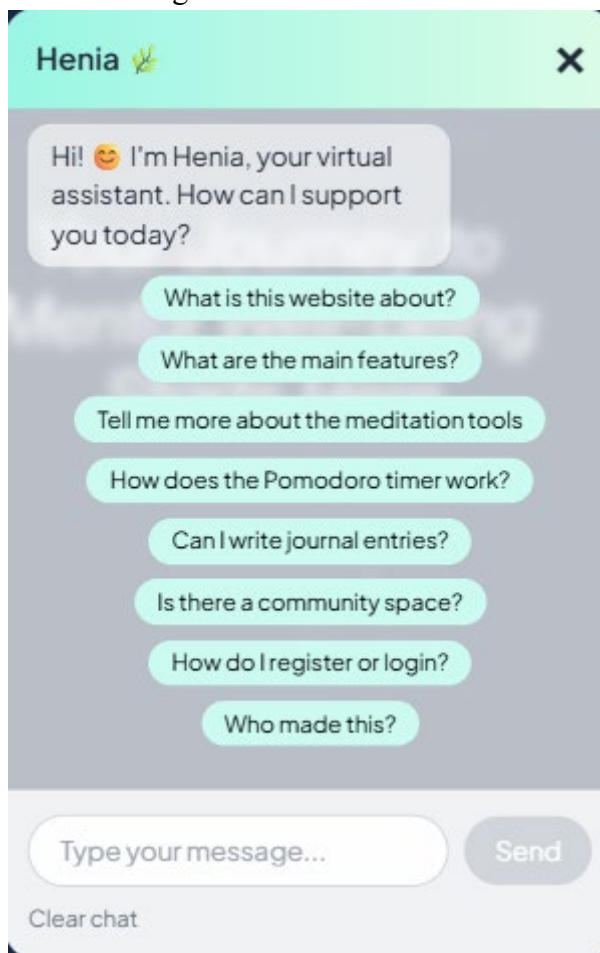
A chatbot az alkalmazás egyik kiegészítő komponense (`utilityComponent`), amely a felhasználók számára egy gyors, barátságos és könnyen elérhető segítséget nyújt a navigációhoz és az alkalmazás funkcióinak megértéséhez. Egy tudatos döntés volt részemről, hogy a chatbot használatához nem szükséges bejelentkezés, így akár egy új felhasználó is azonnal kérdezhet, információt kérhet vagy útbaigazítást kaphat, anélkül, hogy regisztrálnia kellene. Ezzel egy olyan könnyen elérhető támogatást biztosítok, ami javítja az első benyomást, és segít az alkalmazás használatának elsajátításában. A chatbot az egész alkalmazásban ugyanott, a

képernyő jobb alsó sarkában érhető el, ezért a frontend struktúrában a `src/utilityComponents` mappában kapott helyet. Mivel minden oldalon megjelenik, egy különálló komponensként viselkedik, és független a többi modultól. A fő cél egy egyszerű, de modern kinézetű, reszponzív és animált chatablak létrehozása volt, amely bármilyen eszközről könnyen kezelhető.

A chatbot neve *Henia* (*fantázianév a mentál-higiénia szópárosból*), amely egy virtuális asszisztens szerepét tölti be. A felhasználó első üzenetként egy barátságos köszöntést kap („Hi! I'm Henia...”), majd innentől kezdve szabadon kommunikálhat vele. A rendszer kétféleképpen képes válaszolni a kérdésekre:

1. Előre definiált promptok alapján,
2. Valós idejű válasz generálással a Google Gemini API 2.5 segítségével.

A chatbot működésének első lépése azoknak a „predefined promptoknak” az ellenőrzése, amelyek egy JSON fájlban találhatók (`chatbotPredefinedPrompts.json`). Ezek olyan gyakori kérdések, amelyekre azonnali, pontos és előre megírt válaszokat ad a rendszer. A promptok között vannak olyanok is, amelyek linket jelenítenek meg az alkalmazáson belül, például átirányítják a felhasználót a meditációs vagy naplózó oldalra. Így a chatbot nemcsakválaszol, hanem valódi navigációs asszisztensként is működik.



7. ábra
Hénia a chatasszisztens

Ha a felhasználó olyan kérdést tesz fel, amely nem illeszkedik az előre definiált promptokhoz, vagy nem található meg a kulcsszavak, vagyis az `allowedKeywords` tömbben, akkor egy elutasító üzenetet kap, vagyis azokra nem tud válaszolni, ami nem az oldal tartalmával kapcsolatos. Ugyan ez az eset áll fenn, ha obszcén, nem helyénvaló tartalmat kérünk tőle vagy

inputot adnánk meg. A modellhez a környezeti változók között tárolt API kulccsal kapcsolódik az alkalmazás. Azért választottam Gemini API-t, mivel mindenféleképpen szerettem volna egy Chatbotot az alkalmazásba, és a Gemini 2.5 jelenleg ingyenesen is használható bizonyos korlátokkal, ami lehetővé tette, hogy erős háttérlogikát építsek a chatbot mögé. A működés során azonban fontos volt számomra, hogy a chatbot ne váljon „szabadon beszélgető” felületté, mivel a célja az, hogy az alkalmazáson belül szolgáljon segítségként. Emiatt, mint már említettem korábban, a modell előtt található egy előszűrés.

- ha a felhasználó trágár vagy nem megfelelő tartalmú üzenetet küld,
- ha teljesen oda nem illő témáról kérdez (például politika, erőszak, egyéb külső témák)

Ekkor a chatbot nem válaszol ezekre, hanem udvariasan elutasítja. Ez a működés szándékosan került így kialakításra, hogy a chatbot a mentális egészség témaköréhez kapcsolódó, biztonságos és támogató válaszokat adjon. Így a chatbot szerepe valóban egy asszisztens, és nem egy általános beszélgetőpartner. A felhasználói élmény szempontjából a chatfelület is sok apró vizuális részletet kapott. A lebegő chatbot ikonra kattintva egy animációval megnyíló ablak jelenik meg, ahol a felhasználó láthatja a beszélgetési előzményeit. A chatbot ikonon egy pulzáló animáció jelzi, hogy bármikor elérhető („ping” framer motion effekt), ami kellemes, figyelemfelkeltő, de nem zavaró. A chat történetben a felhasználói és bot-üzenetek vizuálisan elkülönülnek: a felhasználó üzenetei jobbra igazítva, enyhén színezett háttéren jelennek meg, míg a bot válaszai balra rendezve. A `React Markdown` segítségével pedig a bot képes formázott szöveget is megjeleníteni, például linkeket vagy kiemeléseket. A felhasználó a beszélgetés során bármikor törölheti a chat előzményt, ilyenkor a chatbot egy kedves üzenettel újraindítja a beszélgetést. Az inputmező Enterrel is működik, és egyszerre figyel a `Shift+Enter` kombinációra, így a több soros üzenetek írása is lehetséges.

Összességében a chatbot egy olyan univerzális modul lett, amely bár minden oldalon jelen van, mégis teljesen elkülönülve működik. Nem igényel bejelentkezést, segíti a felhasználói élményt, gyors tájékozódást biztosít, és modern, AI-alapú interakciókat nyújt. Hénia így nem csupán egy „dísz” az alkalmazásban, hanem egy olyan funkció, amely valódi értéket ad az alkalmazáshoz és a felhasználó támogatva érzi magát a használat során, így a rendszer sokkal „élőbbnek” és személyesebbnek hat.

4.3.3 Meditáció – Meditation Room

A felhasználó a meditáció modulra való navigálás és annak betöltése után találkozik az alkalmazás egyik lényeges részével. Itt rengeteg különálló funkciót kellett összehangolni, ideértve az animációkat, időzítést, hangkezelést, API-hívásokat, valamint a különféle légzéstechnikák logikáját. A célom az volt, hogy a felhasználó egy olyan meditációs szobába érkezzen, ahol minden könnyen kezelhető, letisztult, mégis kellően inspiráló élményt nyújt, és a beépített pozitív üzenettel, ami minden betöltéskor vagy a `Reroll` gombra való nyomás után egy új üzenetet jelenít meg.

A meditációs oldal több különálló komponensből épül fel (`BreathingCircle`, `SettingsPanel`, `ControlButtons`, `PositiveQuotes`, `TechniqueGuide`, `MeditationTips` stb.), és minden komolyabb funkcióhoz külön `hook` tartozik. A fejlesztés során ez rengeteget segített abban, hogy ne egyetlen óriási komponensben kelljen mindent megvalósítani, hanem minden logika saját, elkülönített helyet kapjon. A meditációhoz négy légzésszint tartozik: `Beginner`, `Intermediate`, `Advanced` és `Expert`, amelyek mindegyike más-más ritmust és hosszt követ. A `Beginner` például klasszikus `Box Breathing`, az `Intermediate` a 4–7–8 technikára épül, míg az `Advanced` a koherens légzés mintáját alkalmazza. Az `Expert` szintet a Wim Hof-féle stimuláló, úgynevezett **hiperventilációs** légzés inspirálta, ahol gyors egymás utáni belégzések után lélegzet visszatartás következik. A rendszer nemcsak a hanggal, hanem vizuális animációval és időzítéssel is támogatja ezeket a

ritmusoknak a követését. A légzéstechnikák adatai egy JSON fájlban találhatóak, ezek írják le a belégzés-kilégzés hosszát, a ciklusok számát és a technika rövid leírását, ezért egyszerűen egy lenyíló menüből tudom kiválasztani a beállításokat, melyeket szintén `React State`-el kezelek, így a jövőben, ha változtatni szeretnénk elegendő a JSON objektum tartalmát kibővíteni, módosítani vagy akár törölni belőle adatokat. A légzés animációját a `useBreathing` hook valósítja meg, amely kiszámolja az aktuális fázist (belégzés, kilégzés, visszatartás stb.) és az animáció előrehaladását. A `BreathingCircle` komponens ez alapján növeli vagy csökkenti a kör méretét, ezzel vizuálisan támogatva a ritmus követését.

A vizuális megvalósítás külön kihívást jelentett, mivel a köröket nem lehetett egyszerű CSS-animációkkal mozgatni. A komponens nyolc körből áll, 45°-os eltolással elhelyezve. A forgatást a `getRotation` függvény számolja, amely az animáció előrehaladásától függően adja meg az aktuális szöveget. A körök finom elmozdítását a `getTranslation` biztosítja, így az animáció valódi „lélegző” hatást kelt. A fázisok színét a `getPhaseColor` állítja be, míg a középső instrukciós panelt a `getBreathingInstruction` és `getDurationText` funkciók frissítik. Az Expert szint külön logikát igényelt a gyors belégzések és a visszatartási fázis miatt.

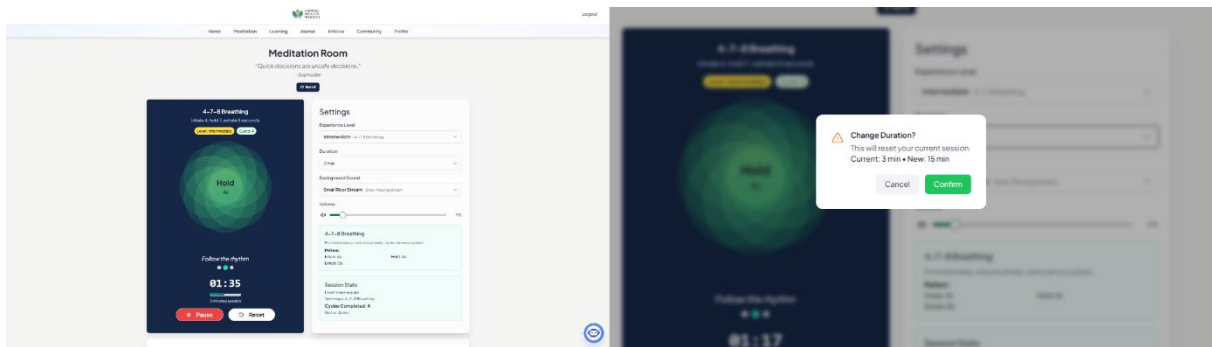
A meditáció során a felhasználó több beállítást megadhat:

- Időtartam (1-60 perc)
- Háttérhang (pl. esőhang, erdei zajok, folyóhang stb.)
- Hangerő + némítás
- Légzésszint

A háttérhangok kezeléséért a `useAudio` hook felel, amely betölti a kiválasztott hangot, kezeli a lejátszást, megállítást, hangerőt, és a némítást. Hasonlóan működik az idő visszaszámlálását kezelő `useTimer` hook is. Ez számolja vissza a meditációt, és a végén lejátszik egy értesítő hangot, hogy jelezze a felhasználónak a folyamat végét. A beállítások megjegyzéséről is gondoskodtam, erre a `usePersistentState` hook szolgál, amely minden beállítást a `localStorage`-ben tárol. Így, ha a felhasználó újratölti az oldalt vagy másnap visszatér, a környezet ugyanabba a konfigurációba tér vissza így nem kell újrabeállítani azokat. A `PositiveQuotes` komponens egy külső API-t hív meg:

```
https://api.quotable.io/random?tags=inspirational|wisdom|peace|mindfulness|healing
```

Ha az API sikeresen válaszol, a felhasználó egy friss, inspiráló idézetet kap a URL-ben felsorolt kategóriák közül (bölcsség, béke stb.). Ha az API nem érhető el, a rendszer automatikusan előre beállított tartalommal helyettesíti azt, így az élmény semmiképpen sem marad hiányos. Az oldal jobb oldalán a `Settings` panel található, ahol a felhasználó minden részletet személyre szabhat. A bal oldalon pedig a tényleges meditációs felület jelenik meg. A légzéskör, az aktuális fázis (például `Breathe In` vagy `Hold`), a visszaszámláló időzítő, valamint a ciklusok száma. A felület időben reagál a felhasználó választásaira, például, ha meditáció közben módosítaná a beállításokat, a rendszer megerősítést kér, hogy valóban megszakítsa-e az aktuális folyamatot.



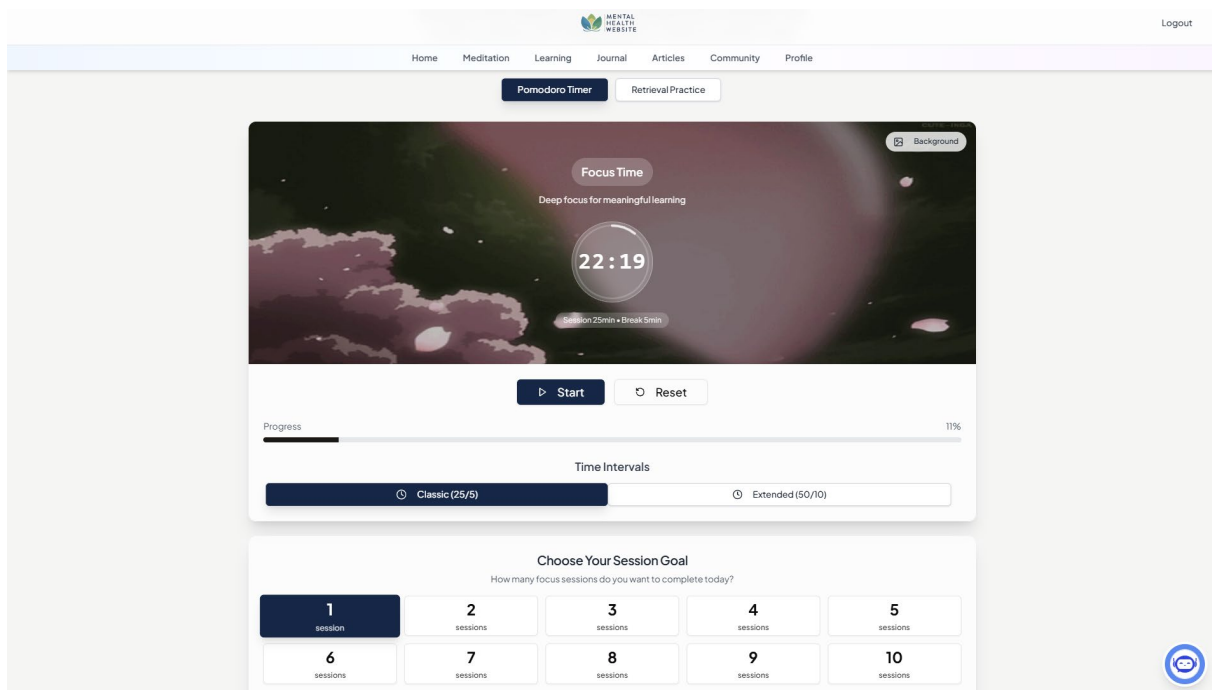
8. ábra
Meditáció beállítás panel és változásértesítés

A meditációs modul így egyszerre technikailag összetett és felhasználóbarát megoldás lett. A háttérben több hook, API-hívás és állapotkezelési logika fut, de a végfelhasználó ebből csak egy egyszerű, nyugodt és áttekinthető felületet lát. A fő cél az volt, hogy a technikai komplexitás ne zavarja a felhasználói élményt.

4.3.4 Tanulás – Learning Hub

A tanulás modulban a fókusz, a produktivitás és a tanulási folyamatnak a támogatása került előtérbe. A cél itt is az volt, hogy a felhasználó egy egyszerűen kezelhető, vizuálisan kellemes, mégis technikailag letisztult felületet kapjon, ahol kiválaszthatja a számára legalkalmasabb tanulási módszert.

A modul két fő technikát kínál. Pomodoro és Retrieval Practice, amelyek mindegyike hasonlóan, a meditáció modulból ismeretes módon, külön logikával, saját hookokkal és komponensstruktúrával rendelkezik, valamint több kisebb egységből áll, és minden fontosabb funkció mögött külön hook kapott helyet (`usePomodoro`, `useRetrievalPractice`, `useAudio`). Pomodoro technikában lehetőség van a fókuszidő és pihenőidő testreszabására, melyre kétféle lehetőség áll rendelkezésre `Classic` (25/5) és `Extended` (50/10). A Pomodoro módszer népszerű fókusztechnikának számít, amely rövid, intenzív munkaszakaszokra és rövid szünetekre bontja a tanulási időt. A felhasználó a `TechniqueSelector` komponenssel választhatja ki a Pomodoro módot, és a hozzá tartozó beállítások azonnal megjelennek. A rendszer két előre konfigurált időszámítást kínál, mely paraméterbeállítások hasonlóan egy JSON fájlból érkeznek. A kiválasztott időintervallumot a `usePomodoro` hook vezérli, amely kezeli az időzítést, a visszaszámlálást és a szakaszváltást (munka → szünet). A hangkezeléshez külön `useAudio` hook készült, amely biztosítja a háttérzenék lejátszását és az értesítő hangokat. A Pomodoro modul lehetőséget ad a felhasználónak arra, hogy napi tanulási célokat határozzon meg. A `SessionTargetSelector` komponenssel beállítható, hány fókusz-szakaszt (session) szeretne teljesíteni (1–10 között). Ez motivációs eszközként is funkcionál, hiszen a modul összegyűjti a teljesített köröket, és jelzi, ha a felhasználó elérte az aznapi célját. Mivel ezeket az adatokat `React state` és `localStorage` kezeli, az oldal újratöltése után sem vesznek el. A vizuális élményt tovább erősítik a választható háttérképek. A felhasználó hat különböző, nyugodt hangulatú GIF közül választhat (pl. *Retro Japan*, *Autumn Desk*, *Coastal Beach*). A `PomodoroTimer` komponens a kiválasztott háttéren jeleníti meg a nagy digitális időzítőt, egy progresszív körgyűrűt, a fókusz vagy pihenés állapotát, valamint az aktuális `interval` információit.



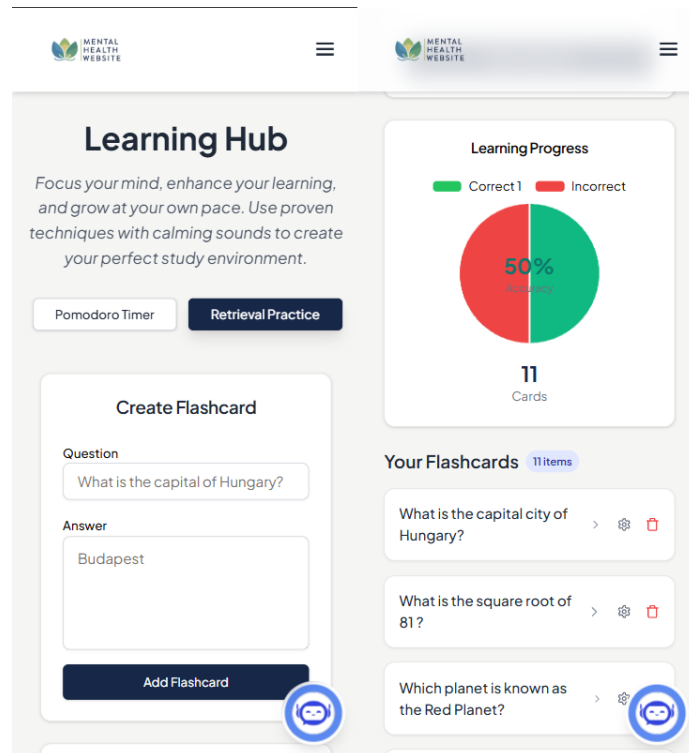
9. ábra
Pomodoro technika modul

A másik tanulási technika az úgynevezett Retrieval Practice, vagyis az aktív visszaidézés. Ez egy rendkívül hatékony tanulási módszer, amely arra épül, hogy saját kérdés-válasz kártyákat hozunk létre, majd ezekből gyakorlunk. A modulban a felhasználó új kártyákat készíthet (Input + Textarea mezőkkel), majd ezeket tesztelheti és a rendszer rögzíti, hogy a válasz helyes volt-e vagy sem.

A `useRetrievalPractice` hook kezeli a kártyák létrehozását, módosítását, törlését, valamint a válaszadás logikáját. Minden kártya elmentésre kerül, így bármikor előhívhatók (a háttértárat szintén a `localStorage` biztosítja).

A modul a válaszokat statisztikává alakítja.

- hány helyes válasz született,
- hány hibás válasz érkezett,
- milyen százalékos pontossággal teljesített a felhasználó.



10. ábra
Tanulási technika választó gomb és Retrieval Practice statisztika

A RetrievalPractice komponens ezekből egy vizuális Pie chart-ot generál, amelyet a Chart.js [16] könyvtár segítségével jelenít meg. A Pie chart megvalósításánál a komponens a felhasználó válaszai alapján minden renderelésnél újraszámolja a szükséges statisztikát. A total, a helyes-hibás arány, valamint az accuracy értékét, amelyet egy egyszerű százalékszámítással ($\text{Math.round}((\text{answers.correct.length} / \text{total}) * 100)$) határoztam meg. A Chart.js-nek átadott pieData struktúra frissül a datasets tömbben, így a grafikon minden egyes válasz után azonnal újrarajzolódik. Vizuálisan kihívást jelentett úgy elhelyezni a középre írt százalékos pontosságot, hogy az ne a Canvas része legyen, ezért egy külön abszolút pozícionált réteget helyeztem a `<Pie />` komponens fölé (`absolute inset-0 flex items-center justify-center`). Mivel a statisztika teljes egészében frontend oldalon, stateből és a `useRetrievalPractice` hookból épül fel, a rendszer valós időben reagál minden interakcióra, így a felhasználó egy gyors és látványos visszajelzést kap. A modul felépítése átlátható, letisztult és könnyen kezelhető.

A legelső opció két gomb, Pomodoro Timer és Retrieval Practice, melyekkel már az elején kiválasztható a technika.

4.3.5 Napló – My journal

A Napló modul már nem pusztán frontend-logikát tartalmaz, hanem a backendel is kommunikál, valamint az adatbázisba is végez műveleteket. A célom az volt, hogy a felhasználó itt saját élményeit, gondolatait vagy napi jegyzeteit rögzítheti, amelyeket a rendszer a backend felé továbbít. A modul CRUD műveleteket valósít meg, így a naplóbejegyzések létrehozása, szerkesztése, törlése és lekérése mind a saját API végpontokon keresztül történik.

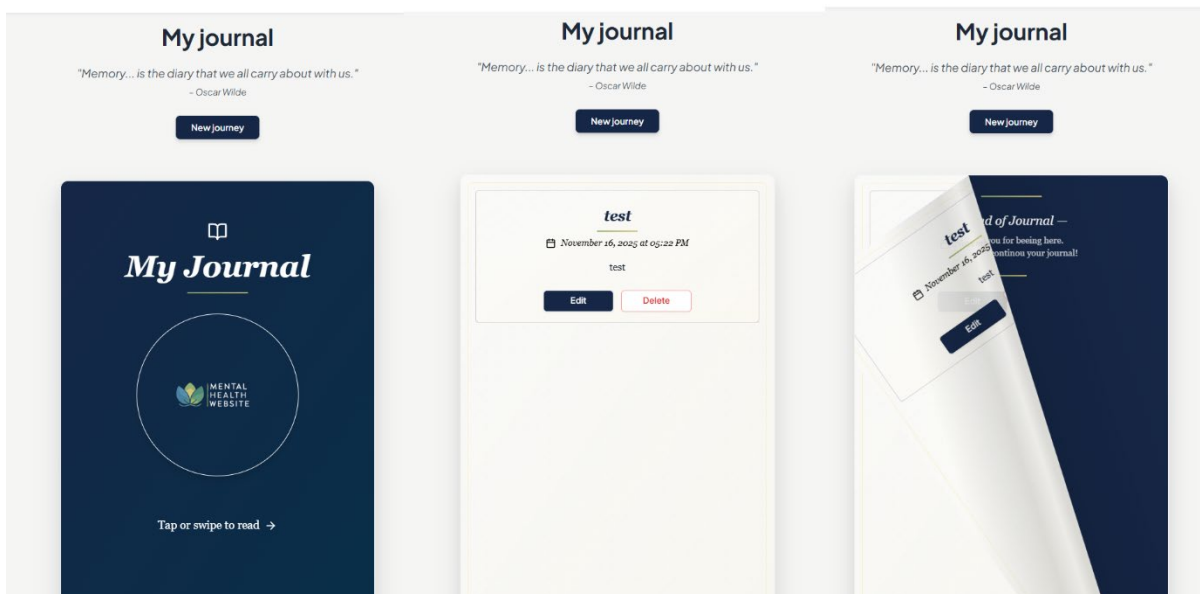
```
await axios.put(`${API_BASE_URL}/${editingJournal.id}`, formData, {
  withCredentials: true,
});
```

A frontend és a backend között axios biztosítja a kapcsolatot, és minden kérés hitelesített (`withCredentials: true`), így csak bejelentkezett felhasználó érheti el a naplóját. A szövegbevitelnél szándékosan nem egy egyszerű `<textarea>` mezőt, hanem a `ReactQuill` rich text editort választottam.

A cél ugyanis az volt, hogy a felhasználó ne csak sima szöveget írhasson, hanem formázhasson, kiemeléseket, listákat, dőlt vagy félkövér részeket állíthasson be, így a naplózás élménye sokkal közelebb kerül a valódi jegyzeteléshez, mint egy egyszerű mező használatával. A `ReactQuill` HTML-t ad vissza, ami viszont biztonsági kockázatot is jelenthet, ezért minden mentés előtt `DOMPurify` tisztítja a bevitt tartalmat. Ezzel megakadályozható bármilyen XSS jellegű script beágyazás, így a napló biztonságos marad, még akkor is, ha a rich text editor HTML-t generál. A tartalom megjelenítéséhez egy különleges megoldást választottam. A bejegyzések interaktív, lapozható `flipbook` formájában jelennek meg. Ehhez a `react-pageflip` könyvtárat használtam, amely lehetővé teszi, hogy a naplót valódi könyvszerű élménnyé alakítsam. A kihívás itt főként vizuális és technikai oldalról jelentkezett, mivel a `flipbook` fix oldal-arányokkal rendelkezik, ezért dinamikusan kellett beállítani és kiszámolni a könyv méreteit a felhasználó képernyőjéhez igazítva. A `useEffect` figyeli az ablakméret változását, és minden töréspontnál különböző dimenziók számolódnak ki:

```
setDimensions({
  width: Math.min(screenWidth - 96, 550),
  height: Math.min(screenHeight - 200, 750),
  minWidth: 400,
  maxWidth: 550,
});
```

Ezt azért kellett így megoldanom, mert a `flipbook` nem skálázza automatikusan a tartalmat, ezért kézzel kellett meghatározni a minimum–maximum méreteket, hogy mobilon, tableten és asztali nézetben is arányos maradjon. A könyv elején egy animált borítóoldal jelenik meg, amint már sokszor említett `Framer Motion`nel kivitelezve, a végén pedig egy lezáró oldal, így az egész interakció valódi napló hangulatot kelt.



11. ábra
Naplóírás, módosítás és lapozás

A REST API-val kommunikáló `useJournal` hook felel a napló teljes üzleti logikájának megfelelő működéséért. Betölti a bejegyzéseket (GET `/api/journals`), kezeli az új bejegyzések létrehozását (POST), frissítését (PUT), valamint a törlést (DELETE). A hook az állapotokat is kezeli, például az űrlap adatokat, a lapozást és a szerkesztés módját. Minden művelet után a `fetchJournals()` újratölti a friss adatokat, így a flipbook azonnal reagál a módosításokra. A megjelenített oldalak mind saját stílusukat kapták. Halvány papírtextúrák, keretek, serif betűtípus és enyhe árnyékolások teszik a naplót esztétikailag is kellemes felületté. Mivel a bejegyzések HTML tartalmat tartalmaznak (a `ReactQuill` miatt), minden oldal a saját `JournalEntry` komponensben renderelődik, így a tartalom teljesen rugalmasan formázható.

Így ez a modul nem pusztán egy egyszerű űrlap és lista kombináció, hanem egy teljes értékű digitális naplóélményt nyújt. Gazdag szövegformázási lehetőségekkel és biztonságos backend kommunikációval biztosítja, hogy bárhol a világban hozzáférhessünk és új bejegyzéseket írhatunk, vagy nosztalgiaként visszaolvashatjuk a régieket. Legyen szó saját számítógépről, mobilról vagy akár egy internetkávézóról, a modul egy vizuálisan különleges flipbook felületen jeleníti meg a tartalmakat.

4.3.6 Cikkek – Articles

A modul feladata, hogy a mentális egészség témájához kapcsolódó cikkeket a felhasználók számára esztétikusan, kategorizálva és jól rendezhető módon jelenítse meg, közben biztosítva az adminisztrátorok számára a cikkek létrehozását, szerkesztését és törlését. A felület egyszerre szolgál tartalomfogyasztásra és tartalomkészítésre, így a jogosultságkezelés kulcsfontosságú szerepet kapott.

A rendszer a cikkírás lehetőségét kizárólag adminisztrátor szerepkörhöz köti. A felületen megjelenő *Write Article* gomb csak akkor válik láthatóvá, ha a bejelentkezett felhasználó adminisztrátori jogosultsággal rendelkezik. Ezt a kliensoldalon a `useArticle()` hook végzi, amely egy profil lekérdezéssel határozza meg a felhasználó szerepét. A háttérben egy egyszerű API-hívás fut le, amely a bejelentkezett user adataiból vizsgálja a `role` mezőt.

```
const res = await axios.get(`${API_URL}/api/users/profile`, {
  withCredentials: true });
if (res.data.role === "admin") setIsAdmin(true);
```

A komponens ennek megfelelően feltételes rendereléssel dönti el, hogy megjelenítse-e a *Write Article* gombot, míg a nem admin felhasználók számára ez az elem teljesen rejtve marad. Ezt a megoldást alkalmaztam, hiszen ez nem csak UX szempontból kifinomult, hanem biztonsági szempontból is lényeges.

The screenshot shows a mobile application interface for writing and publishing an article. At the top, there are two instances of the 'MENTAL HEALTH WEBSITE' logo and three horizontal menu lines. The main content area is divided into several sections:

- Title:** A text input field.
- HTML Editor:** A text area with a toolbar containing options like 'Normal', 'B' (bold), 'I' (italic), 'U' (underline), and icons for bulleted and numbered lists.
- Category:** A list of categories with radio buttons for selection:
 - Mental Health Conditions
 - Trauma & PTSD
 - Neurodiversity & ADHD
 - Therapies & Treatments
 - Finding a Therapist
 - Learning Resources
 - Anxiety, Depression & Mood Disorders
 - Eating & OCD Disorders
 - Emotional Regulation
 - Medication & Psychiatry
 - Mental Health Research
 - Events & Media
- Publish:** A section containing:
 - Status:** Draft
 - Visibility:** Public
 - Upload image or file:** A button with a blue circular icon.
 - Save as draft:** A button.
 - Publish:** A button.

12. ábra
Cikk írása és publikálása, mobil nézetben

A cikkírás során a felhasználó megadhatja a cikk címét, a tartalmat HTML-editor formájában, kategóriát választhat, valamint indexképet is feltölthet. A képfeltöltés backend oldalon a már bemutatott multer közreműködésével történik, az uploadControllerben, a frontend pedig egy FormData objektumot hoz létre, amelyben a fájl adat mellett egyéb metaadatok is továbbíthatók.

Részlet a feltöltésért felelős kódból.

```
const formData = new FormData();
formData.append("file", file);
formData.append("context", "article");
```

A szerver a képet az upload/ könyvtárban tárolja, melyet a saját gépünkről vagy mobileszközünkről, az Upload image or file kattintva tállózzhatunk és a fájlnev visszaküldésével biztosítja, hogy az indexkép megjelenhessen a cikkoldalon.

A cikkek listázása szintén különösen átgondolt módon valósítottam meg. A legtöbb weboldal időrendben listázza az összes bejegyzést, nálam egy sokkal logikusabb elrendezés valósult meg. Kategóriánként csak a három legfrissebb bejegyzés kerül kiválasztásra, majd egy `interleave` algoritmus egymás után fűzi őket úgy, hogy egyik kategória se ismétlődjön túl sűrűn. Ez az algoritmus egy egyszerű mutatókezeléssel járja be kategóriánként a bejegyzéslistákat.

```
const interleavePostsByCategory = (postsByCategory) => { ... };
```

Így a felhasználó minden alkalommal kiegyensúlyozott, frissített és vizuálisan változatos cikkfolyamot lát. A lista paginációt is tartalmaz, amely oldalakra bontja a találatokat, oldalonként öt cikk jelenik meg. A lapozásért felelős `Pagination` komponens a kiválasztott oldalszám alapján újraszámolja a kezdő indexet, majd új cikkblokkokat jelenít meg.

A cikkeket modern, kártyastílusban jelenítem meg a kijelzőn, és ezekben a cikkben található tartalomból, egy rövidített szöveg rész jelenik meg, amelyet a rendszer úgy állít elő, hogy a cikk HTML tartalmát megtisztítva sima szöveggé alakítja. Erre azért van szükség, mert HTML tagek vagy formázások megjelenítése rontaná az áttekinthetőséget.

```
export const getTextFromHTML = (html) => {
  const doc = new DOMParser().parseFromString(html, "text/html");
  return doc.body.textContent || "";
};
```

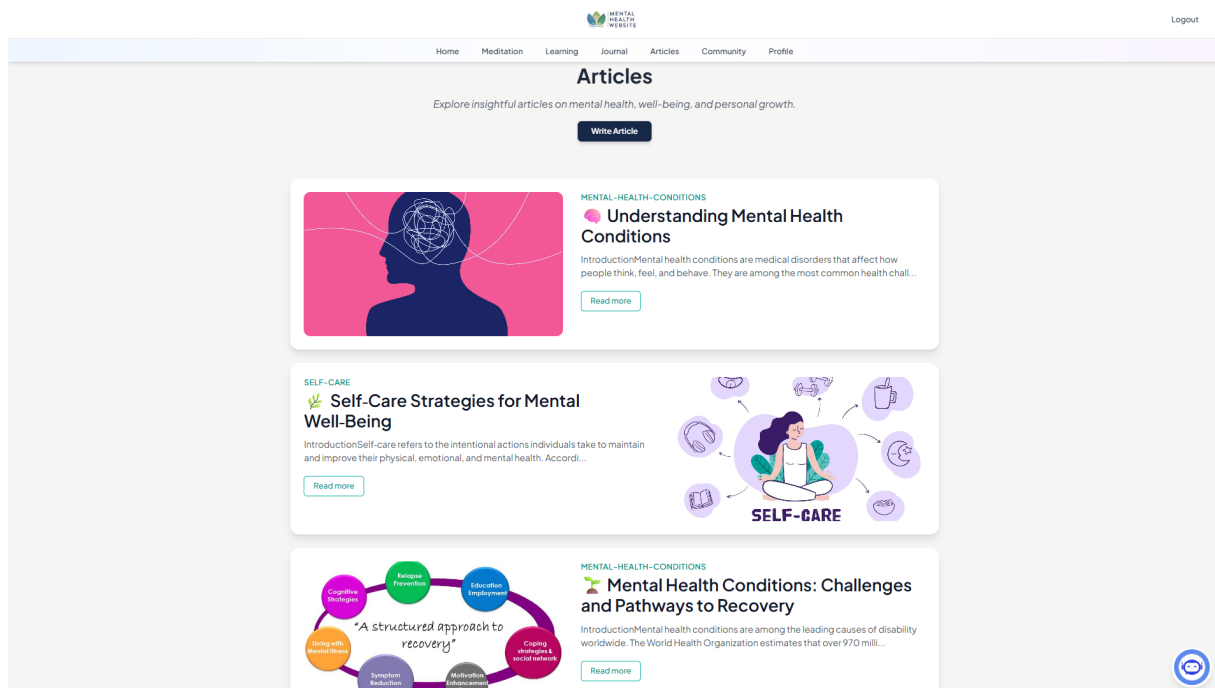
Az `Article` (részletező oldal, egyetlen post nézet) ennél lényegesen gazdagabb struktúrával rendelkezik. Az oldal tetején egy nagy indexkép jelenik meg, amelyet a cikk címe és a szerző metaadatai követnek. A felhasználó neve, profilképe, valamint a bejegyzés relatív dátuma (pl. *Posted 2 days ago*). A cikk tartalma HTML formátumban kerül megjelenítésre, azonban `DOMPurify` használatával előzetesen megtisztítva, így a biztonságot is szem előtt tartja a rendszer.

```
<section dangerouslySetInnerHTML={{ __html:
  DOMPurify.sanitize(post.desc) }}/>
```

A jobb oldali oldalsávban egy automatikusan generált `Kapcsolódó cikkek - Other posts you may like` lista látható, amely ugyanabból a kategóriából ajánl további tartalmakat. A kapcsolódó cikkek lekérdezése külön API-hívással történik, amely kiszűri az éppen olvasott bejegyzést, így az nem jelenik meg kétszer. Ez az ajánlórendszer segít a felhasználóknak további releváns tartalmak böngészésében, ezzel növelve az oldalon eltöltött időt és az olvasási élményt.

A modul zárásaként a cikkek szerzői számára szerkesztési és törlési lehetőségek is elérhetők. Ezek egyszerű ikonokkal jelennek meg a cikkfejléc mellett, és a felhasználó jogosultságától függően jelennek meg és aktiválhatók. A törlés előtt egy megerősítő párbeszédablak jelenik meg, amellyel a rendszer csökkenti a véletlen törlések kockázatát, hasonlóan, meditációs komponensnél, ahol a megszakításra figyelmeztetett. A háttérben a műveletek egy `DELETE /api/posts/:id` végpont meghívásával valósulnak meg.

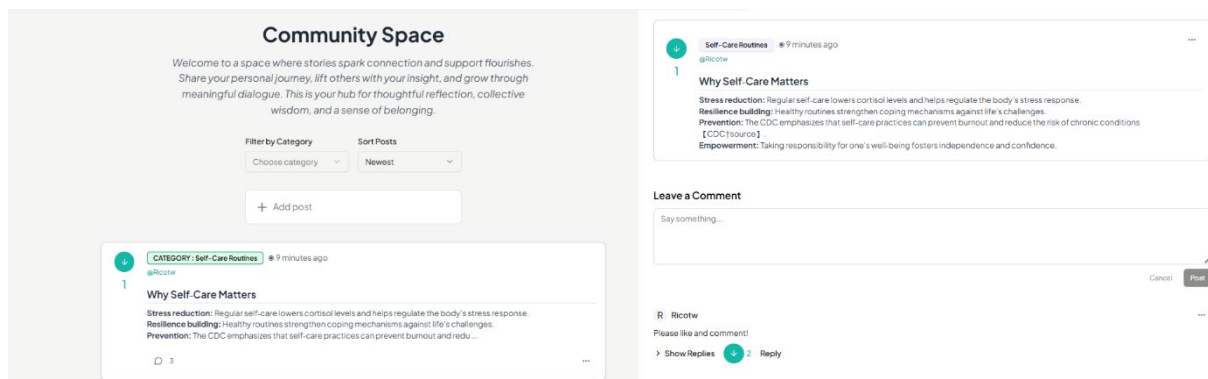
A `Cikkek` modul egy átgondolt, funkciógazdag és technológiailag összetett rendszert alkot, amely teljes körű tartalomkezelést biztosít. A jogosultságkezelés, a képfeltöltés multerrel, a kategorizált cikk keverés, a pagináció, a HTML-tisztítás, valamint a kapcsolódó cikkek dinamikus ajánlása egy modern, felhasználóbarát és biztonságos tartalomfogyasztási élményt hoz létre. Ez a modul jól példázza, hogyan lehet a frontend és backend végpontokat és technológiákat összehangoltan, tiszta architektúrával és hatékony komponens kialakítással alkalmazni egy valós webalkalmazásban.



13. ábra
Cikkek

4.3.7.1 Közösség – Community Space

A Community modul nem egy egyszerű közösségi felületnek kellett lennie, hanem egy teljes, többszintű, Reddit-szerű közösségi rendszer kialakítása volt a cél. A funkcionalitás része a kategóriák szerinti szűrés, rendezés, posztolás, upvote/downvote rendszer, moderáció, valamint a hierarchikus kommentfa, ez utóbbi jelentette a modul fejlesztésének legnagyobb kihívását.



14. ábra
Közösségi fórum, és bejegyzés view

A frontend–backend kommunikációra, mint mindenhol itt is létrehoztam egy külön moduláris réteget, a useCommunity hookot.

Mint mindenhol, ez a hook is gyakorlatilag egy saját API wrapperként viselkedik, és a BASE_URL valamint a dinamikusan összeépített /api/community útvonal köré épül. A lekérések alapját minden esetben ugyanaz a fetch struktúra adja:

```
const res = await fetch(`${BASE_URL}/api/community${endpoint}`, { ...
});
```

A hook a közösségi modulban szükséges összes CRUD funkciót külön, jól elkülönített metódusokon keresztül biztosítja. Ebbe beletartozik a posztok létrehozása, frissítése és törlése, a hozzászólások kezelése, a szavazási logika, valamint a tartalmak jelentése is. A megoldás lényege, hogy a frontend komponensei minden műveletet ugyanazon absztrakciós rétegen keresztül érnek el, ez pedig jelentős mértékben egyszerűsíti a komponensek belső logikáját és csökkenti a kódismétlést.

- createPost, updatePost, deletePost
- upvotePost / downvotePost
- addComment, updateComment, deleteComment
- upvoteComment / downvoteComment
- reportPost, reportComment
- getPosts(category, sort, offset, limit) - **kategória + rendezés + pagináció**
- getCommentsForPost(postId) - **limitált lekérés**

Így a frontend egy egységes, absztrakciós rétegen keresztül kezeli a szerverrel való teljes kommunikációt, ami jelentősen csökkenti a komponensek komplexitását.

A hook a közösségi modulban szükséges összes CRUD funkciót külön, jól elkülönített metódusokon keresztül biztosítja. Ebbe beletartozik a posztok létrehozása, frissítése és törlése, a hozzászólások kezelése, a szavazási logika, valamint a tartalmak jelentése is. A megoldás lényege, hogy a frontend komponensei minden műveletet ugyanazon absztrakciós rétegen keresztül érnek el, ez pedig jelentős mértékben egyszerűsíti a komponensek belső logikáját és csökkenti a kódismétlést.

```
const formatted = data.map((post) => ({
  ...post,
  upvotes: post.upvoteCount,
  comments: post.commentCount,
  author: post.username,
  createdAt: post.created_at,
  isUpvoted: post.isUpvoted
}));
```

Ez garantálja, hogy a frontend komponensek mindenhol azonos adattípusokkal dolgozzanak. A feed ezen kívül kezeli a kategóriaszűrést, a különböző rendezési módokat (pl. legújabb vagy legnépszerűbb), a posztok létrehozását külön modálablakon keresztül, valamint az optimista frissítést is, amelynek lényege, hogy az új bejegyzés a teljes újratöltés előtt azonnal megjelenik.

```
setPosts([formattedPost, ...posts]);
```

A közösségi modul legkomplexebb eleme a kommentrendszer volt. A kommentek nem lineáris módon, hanem többszintű, önmagába ágyazott fa struktúrában jelennek meg. Ennek alapja a parent_comment_id mező: a null érték felső szintű kommentet, míg egy azonosító megadása gyermekkommentet jelent. A backend a kommenteket egy sík listában küldi vissza, így a fa összeállítása teljes egészében a frontend feladata volt. Ehhez először egy parent-children struktúrát kellett előállítani.

```
const repliesMap = comments.reduce((map, comment) => {
  const parentId = comment.parent_comment_id;
  if (parentId !== null) {
    if (!map[parentId]) map[parentId] = [];
    map[parentId].push(comment);
  }
  return map;
}, {});
```

Ez a megoldás $O(n)$ műveletigényű, így nagy mennyiségű komment esetén is gyorsan és skálázhatóan működik. A megjelenítésnél külön állapot tárolja, hogy egy adott szinten hány válasz legyen látható, illetve mikor szükséges „+ N more” jelzéssel további válaszok megjelenítését engedélyezni.

```
const [visibleRepliesCount, setVisibleRepliesCount] = useState({});
```

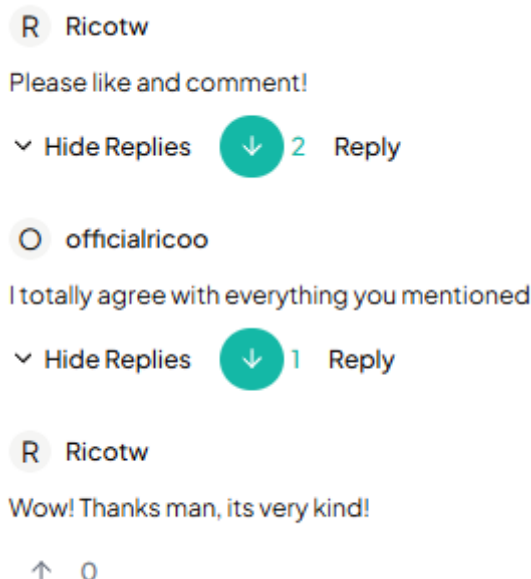
Ez az állapot oldotta meg azt a problémát, hogy a mélyebb kommentláncok (akár 20+ szint) ne okozzanak hosszú, áttekinthetetlen „scroll-falakat”. A `renderReplies(...)` függvény felel a rekurzív megjelenítésért és a fokozatos betöltésért.

```
{areRepliesVisible && renderReplies(comment.id, 0)}
```

A teljes rendszer több iteráción keresztül vált stabilabbá, mivel kezdetben előfordult hibás indentálás, duplikált megjelenítés, rosszul kezelt rekurzió vagy scroll-pozíció ugrálás is. A jelenlegi struktúra viszont már teljesítménybarát és megbízható rekurzív renderelést biztosít. A kommentekhez kapcsolódó műveletek – szavazás, válaszadás, törlés és jelentés különálló komponensekben valósulnak meg. Az optimista `upvote` például először lokálisan módosítja a számokat és a felhasználói élmény érdekében a szavazást optimistán frissítem.

```
setIsUpvoted(newState);
setUpvotes(prev => newState ? prev + 1 : prev - 1);
```

Majd elküldi a backendnek, és hiba esetén visszaállítja az eredeti értékeket. A válaszadás egy dedikált űrlappal történik, amely közvetlenül az adott komment alatt jelenik meg. A gyermekcomment automatikusan bekerül a megfelelő helyre, és szükség esetén a rendszer a szülő komment ágát automatikusan kibontja, hogy a válasz látható legyen.



15. ábra
Comment fa

A reply űrlap (`CommunityCommentForm`) közvetlenül a szülő alatt jelenik meg, a gyerek komment automatikusan bekerül a listába, és ha szükséges, a rendszer automatikusan kibontja a szülő válaszait. A modul technikai sajátosságai a normalizált posztstruktúra, kategória szűrés és rendezés, optimista frissítések, rekurzív kommentfa, fokozatos rendereléssel, upvote/downvote rendszer, moderáció (report), teljes CRUD posztokra és kommentekre, tiszta komponensszétválasztás (`Feed`, `PostCard`, `CommentTree`, `CommentCard`, `Formok`) Ez a modul gyakorlatilag egy modern, mini Reddit, a teljes infrastruktúrájával együtt.

4.3.7.2 Profil

Mivel a profiloldal mind funkcionalitásában, mind bonyolultságában jóval egyszerűbb, mint az összes többi eddig rész, ezért úgy véltem ennek csak egy alfejezetet szentelek. A felhasználó adatai a `useProfile` hookon keresztül töltődnek be az `/api/users/profile` végpontról.

A hook kezeli:

- `fetchProfile()` – adatok betöltése
- `handleChange()` – lokális állapot frissítése
- `handleUpdate()` – PUT kérés a backend felé
- `handleFileUpload()` – képfeltöltés multipart formátumban
- `isEditing` – űrlap megjelenítése / elrejtése
- `uploading`, `saving`, `loading` flag-ek

A profilkép feltöltése külön hívást igényel, a backend pedig a fájlnevet adja vissza. A frontend ezt azonnal frissíti.

```
handleChange({ target: { name: "img", value: `/upload/${filename}` } });
```

The screenshot displays a user profile for 'Germán Martin' (@Ricotw). The profile header includes a profile picture, name, username, email, and role (admin). Below this are three sections: 'Account Details' (User ID #1, Username @Ricotw, Role admin), 'User Info' (Email Address martingerman991@gmail.com, Birthday April 1, 1999, Member Since November 16, 2025), and 'Security' (Profile Status Active, Verification Verified, Privacy Protected). An 'Edit Profile Information' modal is open in the foreground, containing fields for Full Name (Germán Martin), Email Address (martingerman991@gmail.com), New Password, Confirm Password, and an Upload New Image button. The modal has 'Save Changes' and 'Cancel' buttons at the bottom.

16. ábra
Profil oldal a szerkesztés gombra való kattintása után

A vizuális megvalósítás itt is a Framer Motion felel. A `ProfileHeader`, `ProfileEditForm` és `ProfileInfoCards` komponensek egységes, reszponzív, kártya-alapú megjelenést biztosítanak, `ShadCN` UI elemekkel kombinálva (`Button`, `Input`, `Card`, `Badge`, `Avatar`). A modul célja a megbízható, egyszerű és reszponzív felhasználói adatkezelés volt, nem tartalmaz bonyolult logikát, de jól integrálódik az alkalmazás teljes felhasználói ökoszisztémájába.

5. Továbbfejlesztési ötletek

Bár az alkalmazás jelenlegi formájában teljes mértékben működőképes, a fejlesztés során számos olyan ötlet és jövőbeli lehetőség körvonalazódott bennem, amelyekkel a rendszer tovább bővíthető vagy akár teljesen új funkciók irányába is elmozdítható. Ezek a továbbfejlesztési lehetőségek részben technológiai, részben pedig szolgáltatásbeli bővítések lehetnek, amelyek a felhasználói élményt, a teljesítményt vagy az alkalmazás méretezhetőségét növelnék.

Az egyik legegységesebb irány a jelenleg MySQL alapú adatbázis lecserélése egy felhőalapú megoldásra. Az olyan szolgáltatók, mint például az AWS RDS, a Google Cloud SQL vagy a Supabase, sokkal nagyobb megbízhatóságot és biztonságot tudnak nyújtani a hagyományos, lokálisan futó adatbázisokhoz képest. Egy felhőalapú adatbázis használatával a skálázás is sokkal egyszerűbb lenne, így az alkalmazás később nagyobb forgalmat vagy több felhasználót is képes lenne stabilan kiszolgálni. A migráció megkönnyítené a további fejlesztéseket is, mivel több modern funkció, például automatizált backup, monitoring vagy beépített titkosítás már alaphozzárul elérhető a felhőszolgáltatókban, illetve a weboldal hostolás is egyszerűbben megoldható, hiszen elegendő egy frontendet kiszolgáló szerver, nem kell a backend szolgáltatásokkal is törődni, egyszerűen frontend oldalt implementálni kell a megfelelő SSH és egyéb project beállításokat.

A fájlkezelés és médiatartalmak feltöltése jelenleg lokális tárhelyre történik, ami fejlesztési környezetben megfelelő, de hosszú távon érdemes lenne ezt is felhőre helyezni. Ehhez az AWS S3 tárhelyre gondoltam, amely biztonságos, gyors és nagy mennyiségű adat tárolására alkalmas. A projekt jövőbeli verziójában a feltöltött képek, hanganyagok vagy egyéb fájlok automatikusan az S3-ba kerülhetnének, így elfüggetlenedne a fizikai fájlrendszerrel, ami lényegesen stabilabb működést eredményez. A szolgáltatás bővítésére is számos lehetőség kínálkozik. Az alkalmazás egyik legnagyobb potenciálja abban rejlik, hogy akár előfizetéses rendszer alapja is lehetne. A „subscription only” modell lehetővé tenné, hogy bizonyos funkciók, mint például prémium tartalmak, szakmai cikkek, vezetett meditációk vagy speciális tesztek csak fizető felhasználók számára legyenek elérhetők. Ezzel nemcsak fenntarthatóbbá válna a rendszer, hanem ösztönözné a folyamatos tartalomfejlesztést is. Egy másik izgalmas bővítési irány az élő chatroom funkciók beépítése lenne. Tematikus, moderált beszélgetőszobákat lehetne létrehozni például stresszkezelés, önismeret, tanulási technikák vagy éppen szakmai támogatás témakörében, ahol a felhasználók valós időben beszélgethetnének egymással. Ez jelentősen növelné a közösségi élményt, és interaktívvá tenné az alkalmazást. A chatroomokhoz moderációs funkciók is kapcsolódhatnának, például felhasználók némítása, jelentések kezelése vagy adminisztrátori jogosultságok kiosztása. Ezek segítenék a biztonságos és támogató közösségi környezet fenntartását. A tanulási modulok kapcsán szintén több fejlesztési lehetőség merül fel. Az alkalmazás képes lehetne statisztikákat és fejlődési elemzéseket készíteni a felhasználók aktivitása alapján. Például megjeleníthető lenne, hogy mennyi időt töltöttek elmélkedéssel, milyen gyakran írtak naplót, vagy mely légzéstechnikai gyakorlatokat használták. Ezekből a rendszer személyre szabott javaslatokat is adhatna, ami már egy teljesen új szintű felhasználói élményt biztosítana. A mentális egészség

témaköréhez kapcsolódva továbbá beépíthetők lennének pszichológiai vagy mentálhigiénés tesztek is. Ezek nem diagnosztikai eszközök, hanem önértékelő jellegű kérdéssorok lennének, amelyek segítségével a felhasználó képet kaphatna aktuális állapotáról, stressz-szintjéről vagy érzelmi mintáiról. Az eredmények természetesen összhangban lennének az adatvédelmi szabályokkal, és szükség esetén akár anonim módon is kitölthetők lennének.

Úgy gondolom, hogy ezek a fejlesztési ötletek jól mutatják, hogy az alkalmazás jelenlegi állapota egy erős alap, amelyre később további modulok, funkciók és szolgáltatások építhetők. A projekt technológiai felépítése a React alapú frontend, a moduláris Node.js + Express alapú backend, valamint a rugalmas architektúra lehetővé teszi, hogy a rendszer hosszú távon is könnyen alakítható legyen. A továbbfejlesztések nemcsak a funkcionalitást bővítenék, hanem a felhasználói élményt és a szolgáltatás értékét is jelentősen növelnék.

Irodalomjegyzék

- [1] Hivatalos Vite dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://vitejs.dev/>
- [2] Hivatalos React dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://react.dev/>
- [3] Hivatalos TailwindCSS dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://tailwindcss.com/docs>
- [4] Hivatalos Node.js dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://nodejs.org/en/docs>
- [5] Hivatalos Express dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://expressjs.com/>
- [6] Hivatalos MySQL és phpMyAdmin dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://dev.mysql.com/doc/>
<https://www.phpmyadmin.net/>
- [7] JWT dokumentáció és npm csomag leírás *(Utoljára megtekintve: 2025. december)*
<https://jwt.io/>
<https://www.npmjs.com/package/jsonwebtoken>
- [8] Hivatalos Axios dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://axios-http.com/docs/intro>
- [9] EmailJS dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://www.emailjs.com/docs/>
- [10] Nodemailer dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://nodemailer.com/about/>
- [11] Material UI dokumentáció *(Utoljára megtekintve: 2025. december)*
<https://mui.com/material-ui/getting-started/overview/>
- [12] shadcn/ui dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://ui.shadcn.com/>

- [13] Framer Motion dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://motion.dev/>

- [14] AOS (Animate on Scroll) dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/michalsnik/aos>

- [15] Particles.js dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/VincentGarreau/particles.js>

- [16] react-chartjs-2 dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://react-chartjs-2.js.org/>

- [17] React Quill dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/zenoamaro/react-quill>

- [18] React Markdown dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/remarkjs/react-markdown>

- [19] Moment.js dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://momentjs.com/docs/>

- [20] DOMPurify dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/cure53/DOMPurify>

- [21] js-cookie dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/js-cookie/js-cookie>

- [22] cookie-parser dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://expressjs.com/en/resources/middleware/cookie-parser.html>

- [23] CORS dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/cors>

- [24] mysql2 dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/mysql2>

- [25] bcrypt dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/kelektiv/node.bcrypt.js>

- [26] @react-oauth/google dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/@react-oauth/google>

- [27] google-auth-library dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/google-auth-library>

- [28] Multer dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/expressjs/multer>

- [29] jsonwebtoken dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://github.com/auth0/node-jsonwebtoken>

- [30] jwt-decode dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/jwt-decode>

- [31] dotenv dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/package/dotenv>

- [32] Gemini API dokumentáció *(Utoljára megtekintve: 2025. december)*

<https://ai.google.dev/gemini-api/docs>

- [33] Node Package Manager *(Utoljára megtekintve: 2025. december)*

<https://www.npmjs.com/>

- [34] React Router *(Utoljára megtekintve: 2025. december)*

<https://reactrouter.com/>

Nyilatkozat

Alulírott Germán Martin üzemmérnök-informatikus Bprof szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, üzemmérnök-informatikus Bprof diploma megszerzése érdekében. Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Dátum (Szeged, 2025. 12. 07)

aláírás

Melléklet

A. Adatbázis táblák, A diagram készítéséhez az Eraser-t használtam.

<https://www.eraser.io/>

